

Curso: Rust e System Design para processamento de arquivos em alto volume

Oct 8, 2026 · @Daniel Freire

Como usar este curso

Você vai sair daqui sabendo projetar e implementar, em Rust 1.99 (edição 2024), um pipeline que recebe milhares de arquivos por dia, processa milhões de linhas e grava tudo no PostgreSQL sem estourar memória, sem duplicar dados e sem perder registros quando algo falha.

É a versão em Rust do curso de processamento de arquivos que já existe em .NET 10 e em Go 1.27: os mesmos dez módulos, o mesmo método **top-down** (o problema e os números, depois a arquitetura, depois cada componente, só então o código) e o mesmo projeto final. O system design não muda. O que muda, e muito, é o modelo de memória: Rust não tem coletor de lixo, e o compilador verifica em tempo de compilação quem é dono de cada valor e por quanto tempo cada referência vive. Isso elimina categorias inteiras de bugs que os outros dois cursos precisavam tratar com disciplina (o buffer reaproveitado do `bufio`, a corrida de dados num lote compartilhado) e cobra em troca uma curva de aprendizado maior.

Pré-requisitos

Sintaxe de Rust: ownership e empréstimos, `struct`, `enum`, `trait`, `Result` e o operador `?`, iteradores e noções de `async / await`. Se ainda não tem, os capítulos 1 a 10 e 16 do *The Rust Programming Language* cobrem o necessário. O Módulo 3 revisita ownership do ponto de vista de desempenho, não de sintaxe.

Exemplo condutor

Arquivos de retorno bancário em layout posicional (CNAB 240/400): linhas de largura fixa, chegada em lotes, processamento exatamente uma vez e tabelas grandes no destino. O worker deste curso consome as mesmas mensagens e grava nas mesmas tabelas que os workers em .NET e Go, e poderia dividir a fila com eles.

Stack usada

- Rust 1.99, edição 2024, com `cargo`
- `tokio` (runtime assíncrono, canais, tarefas) e `tokio-util` (`CancellationToken`)

- PostgreSQL com `tokio-postgres` (incluindo `COPY` binário), `deadpool-postgres` (pool) e `sqlx` como alternativa para consultas verificadas em tempo de compilação (Módulo 7)
- AWS SDK para Rust (S3 e SQS)
- `thiserror` e `anyhow` para erros
- `tracing` e OpenTelemetry para observabilidade; `criterion` para benchmarks

O mesmo problema em três plataformas

Conceito	.NET 10	Go 1.27	Rust 1.99
Gerenciamento de memória	GC geracional	GC concorrente não geracional	Ownership, sem GC
Janela sobre bytes	<code>ReadOnlySpan<byte></code>	<code>[]byte</code> (fatia)	<code>&[u8]</code> (empréstimo com tempo de vida)
Leitura em streaming	<code>System.IO.Pipelines</code>	<code>bufio.Scanner</code>	<code>tokio::io::BufReader</code> + <code>read_until</code>
Reaproveitar buffers	<code>ArrayPool<T></code>	<code>sync.Pool</code>	Reusar o <code>Vec</code> (<code>clear()</code> mantém a capacidade)
Concorrência	<code>async/await</code> + <code>ThreadPool</code>	Goroutines	<code>async/await</code> + tarefas do tokio
Canal com limite	<code>Channel<T></code> bounded	<code>chan T</code> com buffer	<code>async_channel::bounded(n)</code>
Grupo de tarefas	<code>Task.WhenAll</code> + cancelamento	<code>errgroup</code>	<code>JoinSet</code> + <code>CancellationToken</code>
Erros	Exceções	Valores <code>error</code>	<code>Result<T, E></code> + <code>?</code>
Escrita em massa	<code>BeginBinaryImportAsync</code>	<code>CopyFrom</code> (pgx)	<code>BinaryCopyInWriter</code> (tokio-postgres)
Corrida de dados	Encontrada em teste, se tiver sorte	<code>go test -race</code>	Erro de compilação

A última linha resume o argumento de Rust para este tipo de sistema: o pipeline do Módulo 5, com um leitor e vários escritores trocando lotes, simplesmente não compila se dois deles puderem tocar o mesmo lote ao mesmo tempo.

Mapa dos módulos

Módulo	Pergunta que responde	O que você entrega
1. O problema	Quanto volume, com que prazo, com que garantias?	Uma ficha de requisitos com números
2. Arquitetura	Quais peças existem e como conversam?	O diagrama de referência e a tabela <code>arquivo</code>
3. Memória	Quem é dono de cada byte, e quando ele é liberado?	Um parser de bytes sem alocação e o runtime configurado
4. I/O de arquivos	Como ler 2 GB sem carregar 2 GB?	Um leitor em streaming que entrega lotes
5. Concorrência	Como paralelizar sem afogar o banco?	Um pipeline com canais, tarefas e cancelamento
6. Escrita em massa	Como inserir 1 milhão de linhas em segundos?	Um escritor com <code>COPY</code> binário e upsert
7. Leitura em massa	Como ler tabelas grandes sem travar nada?	Streams de linhas e keyset
8. Confiabilidade	O que acontece quando algo quebra no meio?	Lease, retries, DLQ, outbox e reconciliação
9. Observabilidade	Como sei que está rápido e saudável?	<code>tracing</code> , métricas, perfis e benchmarks
10. Escala	Como crescer de 1 para N workers?	Deploy com autoscaling por fila
Projeto final	Tudo junto funciona?	O worker completo, testado sob carga

Sugestão de ritmo: um módulo por semana, e duas para o Módulo 3 se ownership ainda não for natural. Os Módulos 3, 4 e 5 são os que mais diferem dos cursos de .NET e Go.

Módulo 1 — O problema: anatomia de um pipeline de alto volume

Antes de escrever código, você precisa de três números: quantas linhas por segundo o sistema sustenta no pico, em quanto tempo um arquivo precisa estar processado e qual garantia de entrega o negócio exige. Rust é rápido, mas velocidade da linguagem não substitui dimensionamento: o gargalo deste sistema quase nunca é a CPU do worker.

1.1 Onde o jeito ingênuo quebra

O jeito ingênuo em Rust é: `std::fs::read_to_string`, `.lines()`, um `.to_string()` e um `.parse::<i64>()` por campo e um `INSERT` por linha. Ele quebra nos mesmos três pontos de sempre:

1. **Memória:** `read_to_string` coloca o arquivo inteiro numa `String`, e ainda falha se o arquivo não for UTF-8 válido (arquivos bancários em ISO-8859-1 com acentos fazem exatamente isso).
2. **Banco:** um `INSERT` por linha é uma ida e volta de rede por linha. Com 1 ms de latência, 1 milhão de linhas custam mais de 16 minutos só de espera, e nenhuma otimização de compilador muda isso.
3. **Falha:** se o processo cai na linha 800 mil, não há como saber o que foi gravado.

E Rust tem duas armadilhas próprias para quem está começando:

- **`.clone()` para calar o compilador.** Quando o verificador de empréstimos reclama, clonar resolve o erro e esconde uma alocação. Um `String` clonado por campo por linha são milhões de alocações, e o programa sem GC fica mais lento que o com GC bem escrito. O Módulo 3 mostra como escrever o mesmo código emprestando em vez de copiando.
- **`tokio::spawn` sem limite.** Criar uma tarefa por linha é tão barato quanto criar uma goroutine, e tão errado quanto: milhões de tarefas disputando um pool de 20 conexões. Toda concorrência é explicitamente limitada (Módulo 5).

1.2 Requisitos que definem a arquitetura

Requisito	Pergunta para o negócio	Por que muda o design
Vazão	Quantas linhas por segundo no pico?	Número de workers e técnica de escrita
Latência ponta a ponta	Em quanto tempo o dado precisa estar disponível?	Lote ou streaming contínuo
Garantia de entrega	Pode perder? Pode duplicar?	Idempotência, transações, checkpoint
Ordenação	A ordem importa?	Limita o paralelismo e exige particionamento
Tamanho dos arquivos	Qual o maior possível, não o médio?	Processar inteiro ou dividir em pedaços

Requisito	Pergunta para o negócio	Por que muda o design
Retenção e auditoria	Por quanto tempo guardar o original?	Armazenamento e rastreabilidade

Projete para o **pior arquivo** e a **hora de pico**.

1.3 Contas de guardanapo

O mesmo cenário dos cursos de .NET e Go, para que os três projetos sejam comparáveis: 5.000 arquivos por dia, média de 40.000 linhas, maior arquivo com 3 milhões de linhas, 60% dos arquivos numa janela de 2 horas e SLA de 15 minutos por arquivo.

São 200 milhões de linhas por dia; na janela de pico, cerca de **16.700 linhas por segundo**, e com margem de 2×, um alvo de 35.000 linhas/s. O maior arquivo exige pelo menos 3.300 linhas/s sozinho. No banco, são 60 GB por dia sem índices, mais de 20 TB por ano: particionamento e retenção entram no projeto desde o primeiro dia (Módulo 7).

1.4 Lei de Little: quantos escritores?

$$L = \lambda \cdot W$$

Com 35.000 linhas/s em lotes de 5.000, são 7 lotes por segundo; se cada lote leva 0,5 s para ser gravado, $L = 3,5$, ou seja, pelo menos 4 escritores concorrentes. Em Rust, L é o número de tarefas escritoras do Módulo 5 e o número de conexões que elas ocupam no pool.

1.5 Onde está o gargalo

- **I/O** (S3, disco): o worker espera bytes. Remédio: buffers maiores, leitura e processamento sobrepostos.
- **CPU** (parsing): núcleos em 100%. Em Rust, com o parser do Módulo 3, este quase nunca é o gargalo; quando for, o remédio é paralelizar o parsing por chunks.
- **Banco** (escrita, locks, índices): o worker espera. Remédio: lotes, **COPY**, staging sem índices e **concorrência limitada**.

Em pipelines de arquivo o gargalo quase sempre é o banco. Uma consequência prática para quem escolhe Rust por desempenho: o ganho do worker em Rust sobre os de .NET e Go aparece em **memória e previsibilidade de latência** (sem pausas de GC, footprint pequeno, mais réplicas por nó), muito mais do que em linhas por segundo gravadas, que são limitadas pelo PostgreSQL.

1.6 Garantias de entrega

- **At-most-once:** pode perder, nunca duplica. Inaceitável para dado financeiro.
- **At-least-once:** nunca perde, pode duplicar. É o que filas e retries entregam.
- **Exactly-once:** o que o negócio quer, obtido como **at-least-once + idempotência** (Módulo 8).

Exercício do Módulo 1

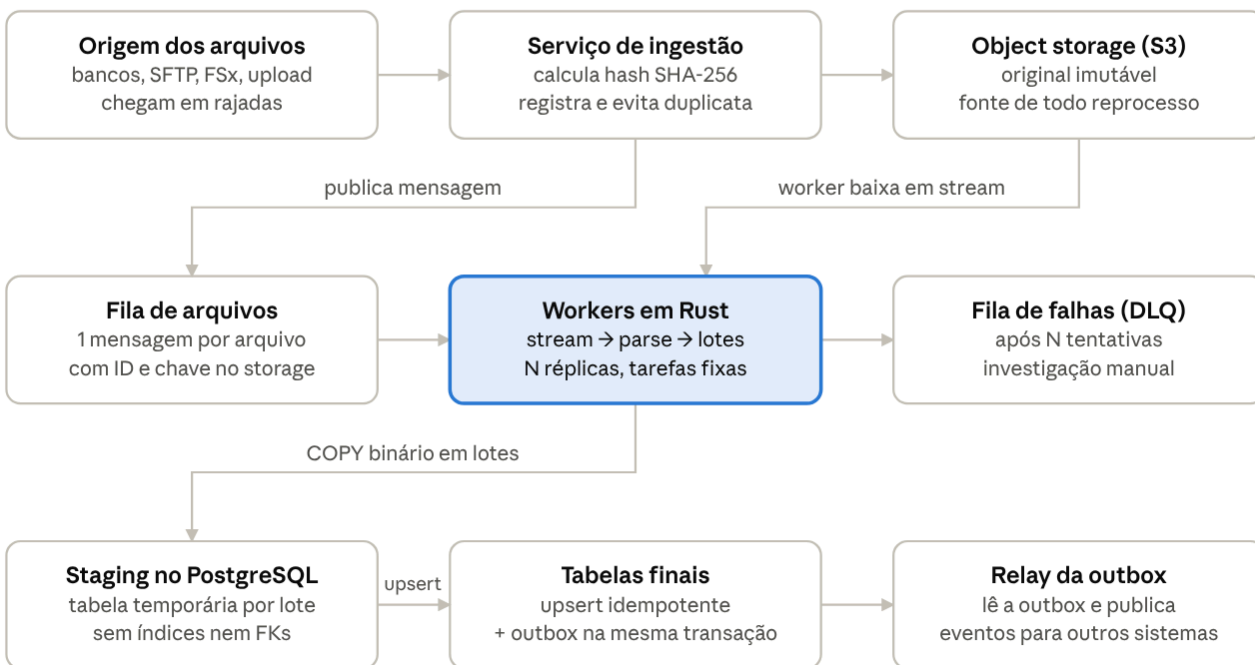
Preencha a ficha de requisitos do seu cenário: arquivos por dia, linhas por arquivo (média e máximo), janela de pico, SLA, garantia de entrega e retenção. Calcule linhas por segundo no pico, volume diário e escritores pela Lei de Little. Depois escreva a versão ingênua (`read_to_string + lines + to_string` por campo + `INSERT` por linha), rode-a num arquivo de 200 mil linhas com `cargo run --release` e anote tempo e pico de memória. Repita com o arquivo salvo em ISO-8859-1 com um nome acentuado e observe o erro de UTF-8.

Módulo 2 — System design: a arquitetura de referência

A arquitetura é a mesma dos cursos de .NET e Go, porque responde ao problema e não à linguagem: separar **receber** de **processar**, guardar o original intacto, distribuir o trabalho por uma fila, processar em workers sem estado que escrevem em lotes numa staging e consolidam com uma operação idempotente, e propagar eventos por uma outbox.

O arquivo entra uma vez, é processado em lotes e vira eventos pela outbox

Arquitetura de referência · o worker em Rust em destaque é o foco dos Módulos 3 a 7



arquitetura de referência · 9 componentes, 3 camadas

A primeira linha recebe e guarda, a segunda distribui, a terceira grava e propaga. Cada seta é um ponto de falha, e cada componente existe para que a falha seja recuperável.

2.1 Os seis princípios

1. **Separe ingestão de processamento:** rajadas são aceitas em segundos e drenadas no ritmo do banco.
2. **O original é imutável e fica guardado:** todo bug vira "corrigir e reprocessar".
3. **A unidade de trabalho é explícita:** o arquivo, ou um pedaço dele; a mensagem aponta, não carrega.
4. **Workers sem estado:** tudo que importa mora no banco.
5. **O banco é protegido:** lotes, concorrência limitada, back-pressure.
6. **Todo passo é idempotente,** o que torna retries seguros.

2.2 Os componentes

Ingestão. Recebe o arquivo, calcula o SHA-256 enquanto grava no storage, registra em `arquivo` com `UNIQUE (cliente_id, sha256)` e só então publica. Pode ser a API de qualquer um dos cursos de API da série.

Object storage. O original, com chave previsível e leitura em stream e por intervalo de bytes.

Fila. Uma mensagem por unidade de trabalho; distribui, reentrega quando um worker some e dá a métrica de atraso para o autoscaling.

Workers em Rust. Leem em stream com `BufReader`, fazem parsing de `&[u8]` sem copiar, agrupam em lotes e gravam com `COPY` binário. Internamente, tarefas do tokio ligadas por um canal com limite (Módulo 5).

DLQ. Mensagens que falham N vezes, para que um arquivo envenenado não trave os outros.

Staging, tabelas finais e outbox. Uma tabela temporária por conexão recebe o `COPY`; um `INSERT ... SELECT ... ON CONFLICT` consolida e grava os eventos na outbox na mesma transação; um relay os publica.

2.3 O ciclo de vida e a tabela `arquivo`

Estado	Significado	Quem muda e para onde
<code>RECEBIDO</code>	Gravado e registrado	Ingestão cria; worker muda para <code>PROCESSANDO</code>
<code>PROCESSANDO</code>	Um worker está com ele, com lease	Worker muda para <code>CONCLUIDO</code> , <code>REJEITADO</code> ou <code>FALHOU</code>
<code>CONCLUIDO</code>	Todas as linhas consolidadas	Final
<code>REJEITADO</code>	Layout inválido; nada aplicado	Final; vai para análise
<code>FALHOU</code>	Erro técnico após N tentativas	Operação pode voltar para <code>RECEBIDO</code>

A tabela é a mesma dos outros cursos, compartilhada por todos os serviços do sistema:

```
CREATE TABLE arquivo (
  id          uuid          PRIMARY KEY,          -- UUID v7,
  gerado pela ingestão
  sequencia   bigint        GENERATED ALWAYS AS IDENTITY UNIQUE,
  cliente_id  uuid          NOT NULL,
  banco       text          NOT NULL,
  nome_original text        NOT NULL,
  sha256      text          NOT NULL,
  chave_storage text        NOT NULL,
  status      text          NOT NULL DEFAULT 'RECEBIDO'
  CHECK (status IN ('RECEBIDO', 'PROCESSANDO', 'CONCLUIDO',
    'REJEITADO', 'FALHOU')),
  tentativas  int           NOT NULL DEFAULT 0,
  worker_id   text          NULL,
```

```

    processando_ate      timestampz NULL,                -- lease
    linhas_processadas  int          NOT NULL DEFAULT 0,
    linhas_rejeitadas   int          NOT NULL DEFAULT 0,
    motivo              text         NULL,
    recebido_em          timestampz NOT NULL DEFAULT now(),
    data_recebimento     date         NOT NULL,
    CONSTRAINT uq_arquivo_conteudo UNIQUE (cliente_id, sha256)
);

CREATE INDEX ix_arquivo_cliente_sequencia ON arquivo (cliente_id, sequencia
DESC);
CREATE INDEX ix_arquivo_pendentes ON arquivo (recebido_em) WHERE status IN
('RECEBIDO', 'PROCESSANDO');
```

Em Rust, o status vira um `enum` com conversão explícita para o texto da coluna, e o compilador obriga cada `match` a tratar os cinco estados:

```

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum StatusArquivo { Recebido, Processando, Concluido, Rejeitado, Falhou
}

impl StatusArquivo {
    pub const fn como_texto(self) -> &'static str {
        match self {
            Self::Recebido => "RECEBIDO",
            Self::Processando => "PROCESSANDO",
            Self::Concluido => "CONCLUIDO",
            Self::Rejeitado => "REJEITADO",
            Self::Falhou => "FALHOU",
        }
    }
}
```

Os IDs v7 vêm do crate `uuid` com a feature `v7` (`Uuid::now_v7()`), ordenados por tempo para que as inserções caiam no fim do índice.

2.4 Arquivos gigantes e alternativas

Para arquivos de milhões de linhas com SLA curto, o fan-out em chunks dos outros cursos: offsets de byte calculados por aritmética (linha k começa em $k \times (\text{tamanho} + \text{quebra})$), uma mensagem por chunk, leitura só do intervalo (`Range` no S3 ou `seek` + `take` num arquivo local) e um fan-in que valida o trailer contra a soma dos chunks.

As alternativas de desenho também são as mesmas: fila gerenciada ou tabela como fila com `FOR UPDATE SKIP LOCKED` (ótima para volume moderado e menos infraestrutura), evento do storage ou polling de pasta, staging + upsert ou upsert direto, arquivo inteiro ou

chunks. Os critérios de escolha estão na tabela do Módulo 2.5 do curso de .NET.

Exercício do Módulo 2

Desenhe a arquitetura para o seu cenário e, para cada seta, escreva o que acontece se o destino ficar fora do ar por 10 minutos: o que se perde, o que se acumula e como o sistema se recupera. Depois crie a tabela `arquivo` num PostgreSQL local e escreva o `UPDATE` condicional de `RECEBIDO` para `PROCESSANDO`. Por fim, acrescente um sexto valor ao `enum StatusArquivo` e veja o compilador apontar cada `match` que ficou incompleto.

Módulo 3 — Memória em Rust: quem é dono de cada byte

Rust não tem coletor de lixo. Cada valor tem exatamente um dono, e a memória é liberada no instante em que o dono sai de escopo. Não há pausas de GC, nem `GOGC`, nem gerações; em troca, cada alocação é uma chamada ao alocador e cada liberação também, e o objetivo do caminho quente continua o mesmo dos outros cursos: **zero alocação por linha**. A grande diferença é que, em Rust, o compilador verifica as regras que nos outros cursos dependiam de disciplina.

3.1 Ownership e Drop: a memória tem dono

Quando um `Vec<RegistroRetorno>` sai de escopo, seu buffer é liberado ali, de forma determinística. Quando ele é movido para outra função ou enviado por um canal, a posse vai junto e o remetente não pode mais usá-lo. Para um pipeline de arquivos, isso significa:

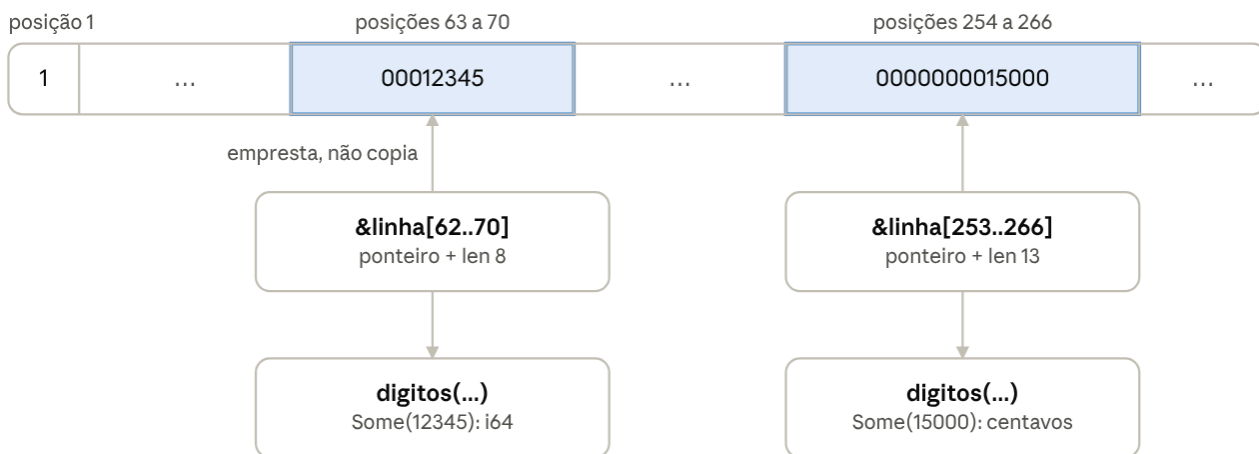
- **Latência previsível.** Sem GC, não há a fração de CPU reservada para marcação nem os *mark assists* do Go, nem coletas de gen2 do .NET. Um lote leva o mesmo tempo no primeiro minuto e na quinta hora.
- **Memória igual ao que está vivo.** O processo não precisa de folga para o GC trabalhar, então o footprint é basicamente buffers + lotes em voo. Não existe `GOMEMLIMIT` porque não há nada para limitar.
- **Alocação ainda custa.** `malloc` e `free` custam dezenas de nanossegundos e, com várias threads, podem disputar locks do alocador. Um programa Rust que aloca por linha pode ser mais lento que um Go bem escrito.

3.2 Empréstimos: o Span com garantia

Um `&[u8]` é um ponteiro mais um tamanho, exatamente como um `ReadOnlySpan<byte>` do .NET ou uma fatia de Go. Fatiar (`&linha[62..70]`) não copia nada. A diferença é o **tempo de vida**: o compilador sabe de qual buffer a fatia foi emprestada e não permite que ela seja usada depois que o buffer mudar ou deixar de existir.

Um `&[u8]` é uma janela que o compilador não deixa sobreviver ao buffer

Uma linha CNAB de 400 bytes dentro do Vec reaproveitado pelo `read_until` · fora de escala



Guardar a fatia além do próximo `read_until` não compila: o empréstimo termina antes do buffer mudar

empréstimos sobre uma linha CNAB · 2 janelas, nenhuma cópia

No curso de Go, a armadilha 1 do Módulo 3.4 era guardar a fatia devolvida pelo `bufio.Scanner` e descobrir depois que ela continha outra linha. Em Rust, o mesmo erro não compila:

```
let mut buf = Vec::with_capacity(512);
let mut guardadas: Vec<&[u8]> = Vec::new();
loop {
    buf.clear();
    if leitor.read_until(b'\n', &mut buf).await? == 0 { break; }
    guardadas.push(&buf[62..70]); // empresta buf...
}
// error[E0502]: cannot borrow `buf` as mutable because it is also borrowed
// as immutable
// ...e o próximo read_until precisaria alterar buf enquanto o empréstimo
// ainda vive
```

O compilador obriga a solução certa: parsear a fatia para valores (`i64`, datas) que não dependem do buffer, ou copiar explicitamente o que precisa ser guardado (`buf[62..70].to_vec()`), e aí a alocação fica visível no código. A armadilha 2 do Go (uma subfatia pequena mantendo um buffer grande vivo) também não acontece por acidente: para a fatia sobreviver, você precisaria manter o buffer inteiro, e isso aparece na assinatura dos tipos.

3.3 Onde as alocações se escondem

Rust deixa as alocações mais visíveis que linguagens com GC, mas não invisíveis. Num caminho quente, desconfie de:

- `String`, `.to_string()`, `.to_owned()`, `format!` e `.clone()` de qualquer coisa que tenha buffer.
- `Vec` crescendo sem `with_capacity` (realocações a cada dobra) e `.collect::<Vec<_>>()` de iteradores longos.
- `Box<dyn Trait>` e `Arc` criados por item.
- `tokio::spawn` por item: cada tarefa é uma alocação com o estado da future.

Para **provar** que o parser não aloca, um teste de integração pode trocar o alocador global por um que conta chamadas:

```
// tests/sem_alocacao.rs: cada arquivo em tests/ é um binário próprio, com
// seu próprio alocador
use std::alloc::{GlobalAlloc, Layout, System};
use std::sync::atomic::{AtomicUsize, Ordering};

struct Contador;
static ALOCACOES: AtomicUsize = AtomicUsize::new(0);

unsafe impl GlobalAlloc for Contador {
    unsafe fn alloc(&self, l: Layout) -> *mut u8 {
        ALOCACOES.fetch_add(1, Ordering::Relaxed);
        unsafe { System.alloc(l) }
    }
    unsafe fn dealloc(&self, p: *mut u8, l: Layout) {
        unsafe { System.dealloc(p, l) }
    }
}

#[global_allocator]
static GLOBAL: Contador = Contador;

#[test]
fn parse_detalhe_nao_aloca() {
    let linhas = gerar_linhas(10_000); // aloca antes da medição
    let antes = ALOCACOES.load(Ordering::Relaxed);
    for (i, l) in linhas.iter().enumerate() {
        cnab::parse_detalhe(1, i as u32).unwrap();
    }
    assert_eq!(ALOCACOES.load(Ordering::Relaxed), antes, "o parser alocou");
}
```

Na edição 2024, operações inseguras dentro de uma `unsafe fn` precisam do próprio bloco `unsafe { }`, por isso os blocos explícitos. Para investigar alocações no programa inteiro, o crate `dhat` faz um perfil de heap que mostra onde cada alocação nasceu.

3.4 O parser sem alocação

O equivalente aos parsers dos cursos de .NET e Go (posições ilustrativas):

```
use chrono::NaiveDate;

pub const TAMANHO_LINHA: usize = 400;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct RegistroRetorno {
    pub nosso_numero: i64,
    pub valor_pago_centavos: i64,
    pub data_ocorrencia: NaiveDate,
    pub numero_linha: u32,
    pub codigo_ocorrencia: u8,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, thiserror::Error)]
pub enum ErroDeLinha {
    #[error("tamanho de linha inválido")]        Tamanho,
    #[error("registro não é de detalhe")]        NaoDetalhe,
    #[error("nosso número inválido")]            NossoNumero,
    #[error("código de ocorrência inválido")]    Ocorrencia,
    #[error("data inválida")]                    Data,
    #[error("valor pago inválido")]              Valor,
    #[error("trailer inválido")]                 Trailer,
}

pub fn parse_detalhe(linha: &[u8], numero_linha: u32) ->
Result<RegistroRetorno, ErroDeLinha> {
    // Converter para um array de tamanho fixo valida o tamanho uma vez
    // e permite ao compilador eliminar as checagens de limite dos índices
    abaixo
    let linha: &[u8; TAMANHO_LINHA] = linha.try_into().map_err(|_|
ErroDeLinha::Tamanho)?;
    if linha[0] != b'1' {
        return Err(ErroDeLinha::NaoDetalhe);
    }
    // Posições do layout são 1-based e inclusivas: 63 a 70 vira 62..70
    let nosso = digitos(&linha[62..70]).ok_or(ErroDeLinha::NossoNumero)?;
    let ocorrencia =
digitos(&linha[108..110]).ok_or(ErroDeLinha::Ocorrencia)?;
    let data = data_ddmmaa(&linha[110..116]).ok_or(ErroDeLinha::Data)?;
    let valor = digitos(&linha[253..266]).ok_or(ErroDeLinha::Valor)?;
    Ok(RegistroRetorno {
        nosso_numero: nosso,
        valor_pago_centavos: valor,
```

```

        data_ocorrendia: data,
        numero_linha,
        codigo_ocorrendia: ocorrencia as u8,
    })
}

/// Quantidade de registros e valor total do trailer, para a reconciliação
(Módulo 8)
pub fn parse_trailer(linha: &[u8]) -> Result<(u32, i64), ErroDeLinha> {
    let linha: &[u8; TAMANHO_LINHA] = linha.try_into().map_err(|_|
ErroDeLinha::Trailer)?;
    if linha[0] != b'9' {
        return Err(ErroDeLinha::Trailer);
    }
    let quantidade = digitos(&linha[17..25]).ok_or(ErroDeLinha::Trailer)?;
    let total = digitos(&linha[25..39]).ok_or(ErroDeLinha::Trailer)?;
    Ok((quantidade as u32, total))
}

#[inline]
fn digitos(campo: &[u8]) -> Option<i64> {
    campo.iter().try_fold(0i64, |v, &b| {
        let d = b.wrapping_sub(b'0'); // abaixo de '0' dá a volta e fica
maior que 9
        (d <= 9).then(|| v * 10 + i64::from(d))
    })
}

fn data_ddmmaa(c: &[u8]) -> Option<NaiveDate> {
    let dd = digitos(&c[0..2])? as u32;
    let mm = digitos(&c[2..4])? as u32;
    let aa = digitos(&c[4..6])? as i32;
    NaiveDate::from_ymd_opt(2000 + aa, mm, dd) // None para 31/02, mês 13
etc.
}

```

O que observar:

- **&[u8; TAMANHO_LINHA]** . Converter a fatia para referência de array de tamanho fixo valida o tamanho uma vez; a partir dali, os índices constantes são comprovadamente válidos e o compilador remove as checagens de limite.
- **Erros como `enum Copy`** . Devolver `Err(ErroDeLinha::Valor)` não aloca nada; o `thiserror` gera a mensagem para o relatório. É o equivalente tipado dos erros sentinela do Go, com uma vantagem: o `match` sobre `ErroDeLinha` é verificado pelo compilador.

- **try_fold com Option**. O laço de dígitos para no primeiro byte inválido, sem `if` explícito, e compila para o mesmo código que o laço manual.
- **RegistroRetorno é Copy**. Ele só tem números e uma data de 4 bytes, então copiar é copiar 32 bytes; ele pode ir para o lote por valor sem nenhuma referência ao buffer de leitura, que é exatamente o que o compilador exige.
- **chrono::NaiveDate** valida a data (inclusive 31 de fevereiro) e é o tipo que o `tokio-postgres` converte para `date` com a feature de integração do chrono.

3.5 Reaproveitar ou alocar?

Em Rust, `Vec::clear()` zera o tamanho e mantém a capacidade, então um buffer de leitura criado uma vez serve para todas as linhas. Para os lotes que passam do leitor aos escritores, alocar um `Vec::with_capacity(5000)` novo por lote custa **uma alocação a cada 5.000 linhas**, o que é desprezível. Não há motivo para um pool como `sync.Pool` ou `ArrayPool`: a regra é zero alocação por **linha**, não por lote. Se um perfil mostrar o contrário, os escritores podem devolver os `Vec` vazios por um segundo canal, e o leitor os reutiliza.

3.6 O alocador

O alocador padrão é o do sistema. Em imagens Linux com glibc ele é razoável; com **musl** (comum em binários estáticos e imagens Alpine), o `malloc` é bem mais lento sob concorrência. Trocar é uma linha:

```
#[global_allocator]
static GLOBAL: mimalloc::MiMalloc = mimalloc::MiMalloc;
```

`mimalloc` e `jemalloc` (pelo crate `tikv-jemallocator`) são as escolhas mais comuns. Meça com o benchmark do Módulo 9 antes de decidir, mas, se o binário for musl, troque sempre.

3.7 O runtime assíncrono

O tokio, no modo multi-thread, cria uma thread de trabalho por núcleo disponível (`std::thread::available_parallelism`, que no Linux respeita as cotas de CPU do cgroup) e escalona as tarefas sobre elas, como o runtime do Go faz com goroutines. A diferença fundamental é que o escalonamento é **cooperativo**: uma tarefa só cede a thread num `.await`. Disso saem as regras:

- **Nunca bloqueie dentro de código async**. `std::fs`, `std::thread::sleep`, um `Mutex` da biblioteca padrão segurado por muito tempo ou um laço de CPU de segundos param uma thread inteira do runtime. Use `tokio::fs` (que delega a um pool de threads de

bloqueio), `tokio::time::sleep` e, para trabalho de CPU pesado, `tokio::task::spawn_blocking`.

- **Parsear um lote entre dois `.await` é aceitável.** Cinco mil linhas com o parser acima levam dezenas de microssegundos, bem abaixo do que atrapalha o escalonamento.
- **Não segure um `std::sync::MutexGuard` através de um `.await`.** O compilador costuma recusar porque a future deixa de ser `Send`, o que é uma proteção: a tarefa poderia continuar em outra thread com o lock na mão.

O .NET e o Go escondem a maior parte disso (o ThreadPool cresce, o runtime do Go cria threads para chamadas bloqueantes); em Rust é sua responsabilidade, e o `tokio-console` (Módulo 9) mostra tarefas que demoram demais entre dois `.await`.

3.8 O processo do worker

Não há Generic Host. O `main` é assíncrono, traduz os sinais do sistema num `CancellationToken` e devolve erro se algo fatal acontecer:

```
#[tokio::main]
async fn main() -> anyhow::Result<> {
    let config = Config::do_ambiente()?; // falha na subida se faltar algo
    let cancelamento = tokio_util::sync::CancellationToken::new();

    let sinal = cancelamento.clone();
    tokio::spawn(async move {
        let mut term =
            tokio::signal::unix::signal(tokio::signal::unix::SignalKind::terminate())
                .expect("handler de SIGTERM");
        tokio::select! {
            _ = term.recv() => {}
            _ = tokio::signal::ctrl_c() => {}
        }
        sinal.cancel(); // início do desligamento gracioso (Módulo 8.8)
    });

    run(config, cancelamento).await // um Err aqui encerra o processo com
    código 1
}
```

Devolver `anyhow::Result` do `main` imprime a cadeia de erros e sai com código de falha, e o Kubernetes reinicia o pod: o equivalente ao `StopHost` do .NET e ao `os.Exit(1)` do Go. Use `anyhow` nas bordas do programa (o `main`, o laço do worker) e `thiserror` nas bibliotecas internas, onde quem chama precisa distinguir os casos com `match`.

Exercício do Módulo 3

Escreva dois benchmarks com `criterion` sobre 100 mil linhas: um com `std::str::from_utf8 + .parse::<i64>()` por campo, outro com `parse_detalhe`, e compare. Escreva o teste do alocador contador e confirme zero alocações; depois troque um campo do parser por `String::from_utf8_lossy(...).to_string()` e veja o teste falhar. Por fim, escreva de propósito o código de 3.2 que guarda `&buf[62..70]` e leia a mensagem completa do erro E0502.

Módulo 4 — I/O de arquivos de alta performance

O objetivo é o de sempre: ler um arquivo de qualquer tamanho com **memória constante**. Em Rust, com tokio, a peça central é `tokio::io::BufReader` com `read_until`, que lê até a próxima quebra de linha **dentro de um `Vec<u8>` seu**, reaproveitado a cada linha. O leitor deste módulo monta os lotes, envia-os por um canal com limite e, no fim, devolve o contexto do arquivo com os totais do trailer: nenhum estado compartilhado entre tarefas.

4.1 Os três níveis de leitura

Abordagem	Memória	Alocação por linha	Quando usar
<code>std::fs::read + split</code>	Proporcional ao arquivo	Nenhuma, mas tudo vivo de uma vez	Arquivos de configuração
<code>BufRead::lines()</code>	Constante	Uma <code>String</code> por linha, com validação de UTF-8	Volume moderado, texto UTF-8
<code>read_until</code> num <code>Vec</code> reaproveitado	Constante	Zero	Alto volume, bytes em qualquer encoding

`lines()` tem dois problemas para arquivos bancários: aloca uma `String` por linha e devolve erro em bytes que não são UTF-8 válido (qualquer acento em ISO-8859-1). `read_until(b'\n', &mut buf)` trabalha com bytes puros: acrescenta a linha ao final de `buf`, incluindo o `\n`, e devolve quantos bytes leu (zero no fim do arquivo). Como você faz `buf.clear()` antes de cada leitura, a capacidade alocada na primeira linha serve para todas as seguintes.

Um cuidado que não existe no `bufio.Scanner` do Go: `read_until` **não tem limite**. Um arquivo corrompido sem quebras de linha faria `buf` crescer até o tamanho do arquivo. Limite a leitura com `take`, como no leitor abaixo.

4.2 Arquivos locais e pedaços

Para arquivos num volume montado (FSx via SMB, por exemplo), `tokio::fs::File` delega as leituras a um pool de threads de bloqueio, e o `BufReader` reduz o número de chamadas. Para ler só um intervalo (um chunk do fan-out), posicione e limite:

```
let mut arquivo = tokio::fs::File::open(&caminho).await?;
arquivo.seek(SeekFrom::Start(inicio)).await?;
let chunk = arquivo.take(fim - inicio); // o leitor vê só os bytes deste chunk
let contexto = leitor::ler(chunk, arquivo_id, lotes_tx,
ContextoDoArquivo::novo(1000), &cancelamento).await?;
```

4.3 O leitor: linha, parse, lote, envio

```
use tokio::io::{AsyncBufReadExt, AsyncRead, AsyncReadExt, BufReader};
// o canal é um async_channel::bounded, apresentado no Módulo 5
use tokio_util::sync::CancellationToken;

pub const TAMANHO_LOTE: usize = 5_000;
const LIMITE_LINHA: u64 = 1 << 20; // 1 MiB: muito além de qualquer linha CNAB

pub struct Lote {
    pub arquivo_id: Uuid,
    pub numero: u32,
    pub itens: Vec<cnab::RegistroRetorno>,
}

#[derive(Debug, thiserror::Error)]
pub enum ErroDeLeitura {
    #[error("erro de leitura: {0}")]
    Io(#[from] std::io::Error),
    #[error("layout de arquivo inválido: {0}")]
    LayoutInvalido(&'static str),
    #[error("linha {0} passa do limite de 1 MiB")]
    LinhaLongaDemais(u32),
    #[error("os escritores pararam")]
    CanalFechado,
    #[error("cancelado")]
    Cancelado,
}

pub async fn ler<R: AsyncRead + Unpin>(
    origem: R,
    arquivo_id: Uuid,
```

```

    saida: async_channel::Sender<Lote>, // canal com limite e vários
consumidores (Módulo 5)
    mut contexto: ContextoDoArquivo, // o leitor é dono do contexto enquanto
lê...
    cancelamento: &CancellationToken,
) -> Result<ContextoDoArquivo, ErroDeLeitura> { // ...e o devolve no fim
    let mut leitor = BufReader::with_capacity(256 * 1024, origem);
    let mut buf = Vec::with_capacity(512);
    let mut lote = Vec::with_capacity(TAMANHO_LOTE);
    let (mut n, mut numero_lote) = (0u32, 0u32);

    loop {
        buf.clear();
        let lidos = (&mut leitor).take(LIMITE_LINHA).read_until(b'\n', &mut
buf).await?;
        if lidos == 0 {
            break; // fim do arquivo
        }
        n += 1;
        if lidos as u64 == LIMITE_LINHA && buf.last() != Some(&b'\n') {
            return Err(ErroDeLeitura::LinhaLongaDemais(n));
        }
        let linha = buf.strip_suffix(b"\n").unwrap_or(&buf);
        let linha = linha.strip_suffix(b"\r").unwrap_or(linha);

        match linha.first().copied() {
            None => continue,
            Some(b'0') => {

validar_header(linha).map_err(ErroDeLeitura::LayoutInvalido)?;
                continue;
            }
            Some(b'9') => {
                match cnab::parse_trailer(linha) {
                    Ok((quantidade, total)) =>
contexto.registrar_trailer(quantidade, total),
                    Err(e) => contexto.registrar_erro(n, e),
                }
                continue;
            }
            Some(_) => {}
        }

        match cnab::parse_detalhe(linha, n) {
            Ok(registro) => lote.push(registro), // cópia de 32 bytes: nada
aponta para buf

```

```

        Err(e) => {
            contexto.registrar_erro(n, e); // linha inválida não
interrompe a leitura
            continue;
        }
    }

    if lote.len() == TAMANHO_LOTE {
        numero_lote += 1;
        let cheio = std::mem::replace(&mut lote,
Vec::with_capacity(TAMANHO_LOTE));
        enviar(&saida, Lote { arquivo_id, numero: numero_lote, itens:
cheio }, cancelamento).await?;
    }
}

if !lote.is_empty() {
    numero_lote += 1;
    enviar(&saida, Lote { arquivo_id, numero: numero_lote, itens: lote },
cancelamento).await?;
}
contexto.registrar_fim(n);
Ok(contexto)
}

async fn enviar(saida: &async_channel::Sender<Lote>, lote: Lote, c:
&CancellationToken) -> Result<(), ErroDeLeitura> {
    tokio::select! {
        r = saida.send(lote) => r.map_err(|_| ErroDeLeitura::CanalFechado),
// canal cheio = espera
        _ = c.cancelled() => Err(ErroDeLeitura::Cancelado),
    }
}

```

O que o compilador e os tipos garantem aqui, que nos outros cursos dependia de cuidado:

1. **Posse do lote.** `std::mem::replace` tira o lote cheio de `lote` e põe um vazio no lugar; o cheio é **movido** para o canal. Depois do envio, não existe mais nenhuma variável no leitor que dê acesso a ele. O bug "mexer no lote depois de enviá-lo" do curso de Go não é detectado em teste: ele não compila.
2. **Nada aponta para `buf`.** `parse_detalle` devolve um `RegistroRetorno` com valores próprios; se ele guardasse uma fatia de `buf`, o `buf.clear()` da próxima volta não compilaria (Módulo 3.2).

3. **Back-pressure de graça.** `send().await` espera quando o canal está cheio. E se todos os escritores morrerem, o receptor é descartado e o `send` devolve erro imediatamente: o leitor para sem precisar de um sinal separado, ao contrário do Go, em que um envio para um canal sem leitor ficaria preso para sempre.
4. **Cancelamento explícito.** O `select!` sai se o token for cancelado (SIGTERM) enquanto o envio espera. O lote que estava sendo enviado é descartado, o que é correto: o arquivo será reprocessado e a escrita é idempotente.
5. **O contexto volta para quem chamou.** O leitor recebe o `ContextoDoArquivo` por valor e o devolve com os erros de linha e os totais do trailer. Nenhuma outra tarefa o toca, então ele não precisa de `Mutex` nem de `Arc`. A struct em si aparece no Módulo 8.7.

4.4 Lendo direto do S3

O corpo de um `GetObject` do AWS SDK para Rust é um `ByteStream`, que vira um leitor assíncrono com `into_async_read()`:

```
let mut pedido = s3.get_object().bucket(&bucket).key(&chave);
if let Some(c) = &chunk {
    pedido = pedido.range(format!("bytes={}-{}", c.inicio, c.fim - 1)); // só
    o intervalo do chunk
}
let objeto = pedido.send().await?;
let corpo = objeto.body.into_async_read(); // streaming, sem baixar tudo
let contexto = leitor::ler(corpo, arquivo_id, lotes_tx,
ContextoDoArquivo::novo(1000), &cancelamento).await?;
```

Os mesmos dois cuidados dos outros cursos: uma conexão que cai no meio vira `ErroDeLeitura::Io` e o arquivo é reprocessado (seguro porque a escrita é idempotente); e se o processamento for tão lento que a conexão expire enquanto o leitor espera no canal, baixe primeiro para um arquivo temporário e leia de lá.

4.5 Hash enquanto copia

Na ingestão, o SHA-256 sai do mesmo laço que grava o arquivo, com um buffer de 64 KiB reaproveitado (crate `sha2`):

```

let mut hasher = Sha256::new();
let mut bloco = vec![0u8; 64 * 1024];
loop {
    let n = origem.read(&mut bloco).await?;
    if n == 0 {
        break;
    }
    hasher.update(&bloco[..n]);
    destino.write_all(&bloco[..n]).await?;
}
destino.flush().await?;
let sha256 = hex::encode(hasher.finalize());

```

4.6 Escrevendo arquivos

Para arquivos de remessa ou relatórios, formate números sem alocar com o crate `itoa` (formatação de inteiros num buffer na pilha) e escreva por um `BufWriter`:

```

/// Acrescenta v com exatamente n dígitos, zeros à esquerda (campos numéricos
do CNAB)
fn anexar_numero(dst: &mut Vec<u8>, v: i64, n: usize) {
    let mut tmp = itoa::Buffer::new();
    let s = tmp.format(v).as_bytes();
    dst.extend(std::iter::repeat_n(b'0', n.saturating_sub(s.len())));
    dst.extend_from_slice(s);
}

let mut w = tokio::io::BufWriter::with_capacity(256 * 1024,
arquivo_temporario);
let mut linha = Vec::with_capacity(402);
for r in &registros {
    linha.clear();
    linha.push(b'1');
    anexar_numero(&mut linha, r.valor_pago_centavos, 13);
    // ... demais campos
    linha.extend_from_slice(b"\r\n");
    w.write_all(&linha).await?;
}
w.flush().await?; // sem flush, o fim do arquivo fica no buffer

```

Como nos outros cursos: escreva num arquivo temporário e renomeie no fim (`tokio::fs::rename`, atômico no mesmo sistema de arquivos), e no S3 use upload multipart, para que ninguém veja um arquivo pela metade.

4.7 Encoding e compressão

ISO-8859-1 tem uma propriedade conveniente: cada byte corresponde exatamente ao code point Unicode de mesmo valor. Para decodificar os poucos campos de texto que você guarda (o nome do pagador), `campo.iter().map(|&b| char::from(b)).collect::<String>()` é uma conversão exata, sem tabela, e aloca só para esse campo. Se o arquivo puder vir em Windows-1252 (que difere do ISO-8859-1 em 32 posições), use o crate `encoding_rs`.

Para `.gz`, o crate `async-compression` oferece um `GzipDecoder` que embrulha um leitor assíncrono e funciona em streaming. Arquivos `.zip` precisam de acesso aleatório ao índice no fim do arquivo: baixe para um temporário e abra com o crate `zip` dentro de `spawn_blocking`, porque a API dele é síncrona.

Exercício do Módulo 4

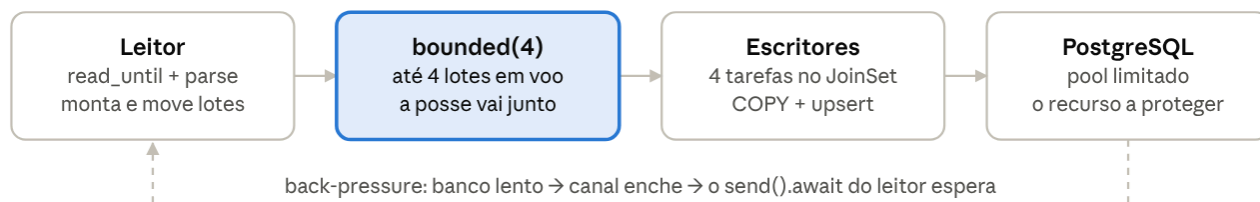
Gere um arquivo sintético de 2 milhões de linhas e leia-o três vezes: com `std::fs::read + split`, com `BufRead::lines()` e com `leitor::ler` (com um consumidor que só descarta os lotes). Meça tempo e pico de memória (`/usr/bin/time -v`). Salve o arquivo em ISO-8859-1 com nomes acentuados e confirme que `lines()` falha e `ler` não. Por fim, remova todas as quebras de linha e confirme que `ler` devolve `LinhaLongaDemais` sem que a memória passe de alguns MB.

Módulo 5 — Concorrência e back-pressure

O pipeline é o mesmo do curso de Go: um leitor que monta lotes, um canal com limite e alguns escritores. Em Rust ele roda sobre tarefas do tokio, os escritores ficam num `JoinSet` e um `CancellationToken` filho do token do processo para tudo de uma vez. E duas propriedades vêm do sistema de tipos: um lote enviado pelo canal muda de dono (ninguém mais pode tocá-lo) e o `panic` de um escritor não derruba o processo.

Um canal com limite faz o banco ditar o ritmo da leitura

Pipeline interno do worker em Rust · escritores num `JoinSet`, todos ligados ao mesmo token de cancelamento



pipeline do worker · 1 leitor, 1 canal, 4 escritores

5.1 Tarefas do tokio

Uma tarefa (`tokio::spawn`) é o equivalente a uma goroutine: barata, escalonada pelo runtime sobre um pool de threads. Três diferenças importam para o desenho:

- **'static e Send**. Uma tarefa pode migrar de thread e viver mais que a função que a criou, então tudo que ela captura precisa ser dono do próprio dado (nada emprestado da pilha de quem chamou) e seguro para mover entre threads. Na prática, você move valores para dentro da tarefa (`async move`) ou compartilha com `Arc`. O compilador recusa o resto, e esse é o lugar onde a maior parte das corridas de dados morre antes de nascer.
- **Concorrência sem tarefa**. Nem tudo precisa de `spawn`. Duas futures na mesma tarefa podem progredir juntas com `tokio::join!` ou `tokio::select!`, sem `'static` nem `Send`. O leitor deste módulo roda assim, na tarefa de quem chama.
- **panic fica na tarefa**. Um `panic` dentro de uma tarefa a encerra e aparece como `JoinError` para quem espera por ela; o processo continua. No Go, um `panic` numa goroutine sem `recover` derruba o processo inteiro.

5.2 O canal: com limite e com vários consumidores

O `tokio::sync::mpsc` é multi-produtor e **um único consumidor**. Para vários escritores lendo do mesmo canal, o caminho mais limpo é o crate `async-channel`, cujo `bounded(n)` é multi-produtor e multi-consumidor e funciona com qualquer runtime. As regras são as que já vimos, com o fechamento vindo da posse:

1. **Sempre com limite**. `async_channel::unbounded` e `tokio::sync::mpsc::unbounded_channel` existem; nunca num pipeline de dados.
2. **O canal fecha quando o último lado de um tipo é solto**. Quando o leitor termina, o `Sender` sai de escopo e os escritores recebem `Err` no `recv`, saindo do laço. Quando todos os escritores terminam, os `Receiver`s saem de escopo e o `send` do leitor falha na hora. Não há `close` para esquecer nem para chamar duas vezes.
3. **Enviar move**. `send(lote)` toma posse do `Lote`; o compilador impede qualquer uso posterior no remetente.

5.3 O orquestrador

O escritor é um trait com método assíncrono. Desde o Rust 1.75, traits podem declarar `async fn` (ou, como aqui, uma função que devolve `impl Future`, o que permite exigir `Send`):

```

pub trait Escriitor: Send + Sync + 'static {
    fn gravar(&self, lote: &Lote) -> impl Future<Output = Result<(),
ErroDeEscrita>> + Send;
}

pub struct Opcoes {
    pub capacidade_lotes: usize, // 4
    pub escritores: usize,      // 4
}

#[derive(Debug, thiserror::Error)]
pub enum ErroDoPipeline {
    #[error(transparent)]
    Leitura(#[from] ErroDeLeitura),
    #[error(transparent)]
    Escrita(#[from] ErroDeEscrita),
    #[error("um escritor entrou em pânico: {0}")]
    Panico(String),
}

pub async fn processar_arquivo<R, E>(
    origem: R,
    arquivo_id: Uuid,
    escritor: Arc<E>,
    opcoes: &Opcoes,
    cancelamento: &CancellationToken,
) -> Result<ContextoDoArquivo, ErroDoPipeline>
where
    R: AsyncRead + Unpin,
    E: Escriitor,
{
    // Token filho: cancelar o processo cancela este arquivo; cancelar este
arquivo não cancela o processo
    let local = cancelamento.child_token();
    let (tx, rx) = async_channel::bounded::<Lote>(opcoes.capacidade_lotes);

    let mut escritores = JoinSet::new();
    for _ in 0..opcoes.escritores {
        let (rx, escritor, local) = (rx.clone(), escritor.clone(),
local.clone());
        escritores.spawn(async move {
            while let Ok(lote) = rx.recv().await {
                if local.is_cancelled() {
                    break;
                }
                escritor.gravar(&lote).await.inspect_err(|_|

```

```

local.cancel())?;
    } // lote é solto aqui: o Vec é liberado
    Ok::<(), ErroDeEscrita>(()))
    });
}
drop(rx); // só os escritores seguram o receptor agora

// O leitor roda nesta mesma tarefa; tx é movido para ele e solto quando
ele termina
let leitura = leitor::ler(origem, arquivo_id, tx,
ContextoDoArquivo::novo(1000), &local).await;
if leitura.is_err() {
    local.cancel(); // não adianta gravar o resto de um arquivo que
falhou
}

let mut primeiro_erro: Option<ErroDoPipeline> = None;
while let Some(resultado) = escritores.join_next().await {
    let erro = match resultado {
        Ok(Ok(())) => continue,
        Ok(Err(e)) => ErroDoPipeline::Escrita(e),
        Err(e) => {
            local.cancel();
            ErroDoPipeline::Panico(e.to_string())
        }
    };
    primeiro_erro.get_or_insert(erro);
}

match primeiro_erro {
    Some(e) => Err(e), // o erro de escrita é a causa real; o "canal
fechado" do leitor é consequência
    None => Ok(leitura?),
}
}

```

O que observar:

- **Propagação de erro.** Um escritor que falha cancela o token local e sai; os outros param no próximo lote; quando todos saem, os receptores são soltos e o `send` do leitor devolve `CanalFechado`. O orquestrador devolve o erro de escrita, não o efeito colateral no leitor.
- **child_token**. Um SIGTERM cancela o token do processo e, com ele, todos os arquivos em andamento. Uma falha num arquivo cancela só o token daquele arquivo.

- **Cancelamento entre lotes.** Um lote que já começou a ser gravado termina (ou falha) e só então o escritor olha o token. Interromper uma transação no meio também seria seguro (o rollback e a idempotência cuidam), mas complica o código sem ganho real.
- **Arc<E> genérico** em vez de `Arc<dyn Escritor>`: traits com métodos assíncronos não são usados como `dyn` diretamente, e o genérico ainda permite ao compilador especializar o código para o escritor real e para o falso dos testes.

5.4 Vários arquivos ao mesmo tempo

Arquivos são independentes, então o laço de consumo limita a concorrência com um semáforo e acompanha as tarefas com um `TaskTracker` (de `tokio-util`), que existe exatamente para "esperar tudo terminar no desligamento":

```
pub async fn consumir(self: Arc<Self>, cancelamento: CancellationToken) {
    let limite = Arc::new(Semaphore::new(self.paralelo)); // 2 arquivos por
    réplica
    let rastreador = TaskTracker::new();

    loop {
        let permissao = tokio::select! {
            p = limite.clone().acquire_owned() => p.expect("o semáforo nunca
            é fechado"),
            _ = cancelamento.cancelled() => break,
        };
        let Some(msg) = self.fila.proxima(&cancelamento).await else {
            break; // None = cancelado durante a espera por mensagem
        };
        let (worker, token) = (self.clone(), cancelamento.clone());
        rastreador.spawn(async move {
            let _permissao = permissao; // devolvida ao semáforo quando a
            tarefa termina
            worker.processar_mensagem(msg, &token).await;
        });
    }

    rastreador.close();
    rastreador.wait().await; // no desligamento, espera os arquivos em
    andamento
}
```

A permissão do semáforo é movida para dentro da tarefa e devolvida quando ela termina, inclusive se terminar com `panic`: o `Drop` roda de qualquer jeito. A mesma conta de conexões dos outros cursos vale: 2 arquivos × 4 escritores = 8 conexões por réplica, mais as de lease e metadados, e é esse o `max_size` do pool (Módulo 10).

5.5 Dimensionando e ordenando

O método de dimensionamento é o do curso de .NET: estimativa pela Lei de Little, teste de carga com 1, 2, 4, 8 e 16 escritores, e o menor número que entrega cerca de 90% da vazão máxima. Quando eventos do mesmo título precisam ser aplicados em ordem, as duas saídas também são as mesmas: N canais com um escritor cada e roteamento por `nosso_numero % N`, ou resolver a ordem no banco ordenando por chave e linha na consolidação.

5.6 Armadilhas de concorrência em Rust

- **Bloquear o runtime:** I/O síncrono, `std::thread::sleep` ou CPU pesada sem `spawn_blocking` para uma thread de trabalho inteira.
- **`std::sync::Mutex` através de `.await`:** quando compila, é um convite a deadlock. Prefira desenhar sem estado compartilhado (canais) ou use `tokio::sync::Mutex` nos raros casos necessários.
- **`select!` e cancelamento de futures:** o ramo que perde é descartado no meio. Algumas operações não são seguras para isso (`read_until` , por exemplo, pode perder bytes já lidos se for descartada). Use `select!` em torno de operações documentadas como *cancel safe* (`send` e `recv` de canais, `cancelled()` , `sleep`), nunca em torno da leitura de uma linha.
- **`spawn` por item e canais sem limite:** os mesmos erros de qualquer linguagem, só que mais fáceis de escrever.
- **Segurar uma conexão do pool durante esperas longas:** a conexão só volta ao pool quando o objeto sai de escopo. Pegue-a o mais tarde possível e solte-a logo.

Exercício do Módulo 5

Monte `processar_arquivo` com um `Escritor` falso que dorme 50 ms por lote e confirme que a memória fica estável. Faça o escritor falhar no lote 10 e verifique que o resultado é `ErroDoPipeline::Escrita`, não o `CanalFechado` do leitor. Depois faça o escritor entrar em `panic` e confirme que o processo continua e o erro volta como `Panico`. Por fim, tente passar ao `spawn` uma referência a um `Vec` da pilha e leia o erro sobre `'static`.

Módulo 6 — Escrita em massa no banco

A receita é do PostgreSQL e vale nas três linguagens: para cada lote, uma transação; dentro dela, o lote vai para uma tabela temporária pelo protocolo `COPY` em formato binário e é consolidado na tabela final por um único `INSERT ... SELECT ... ON`

`CONFLICT`, que grava também os eventos da outbox. Em Rust, o `COPY` binário é o `BinaryCopyInWriter` do `tokio-postgres`, com as conexões vindo de um pool do `deadpool-postgres`.

6.1 As técnicas

Técnica	Idas ao banco por lote de 5.000	Observações
<code>execute</code> por linha, sem transação	5.000 + 5.000 commits	Inaceitável em volume
Vários <code>execute</code> concorrentes na mesma conexão	Poucas (o <code>tokio-postgres</code> faz <i>pipelining</i>)	O servidor ainda executa 5.000 inserções
<code>INSERT ... SELECT FROM unnest(\$1::bigint[], ...)</code>	1	Um parâmetro por coluna; <code>Vec<i64></code> vira <code>bigint[]</code>
<code>COPY ... FROM STDIN (FORMAT BINARY)</code>	1 (stream)	O mais rápido; precisa de staging para fazer upsert

O *pipelining* merece uma nota: o `tokio-postgres` envia várias consultas pela mesma conexão sem esperar as respostas quando as futures são aguardadas juntas (com `try_join_all`, por exemplo). É o equivalente automático do `pgx.Batch` do Go, útil para alguns comandos de metadados numa ida só.

6.2 O modelo de dados

As tabelas são as mesmas dos cursos de .NET e Go:

```
CREATE TABLE retorno_detalhe (
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  arquivo_id  uuid      NOT NULL REFERENCES arquivo (id),
  numero_linha int      NOT NULL,
  nosso_numero bigint   NOT NULL,
  codigo_ocorrencia smallint NOT NULL,
  data_ocorrencia date    NOT NULL,
  valor_pago_centavos bigint NOT NULL,
  gravado_em  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT uq_retorno_linha UNIQUE (arquivo_id, numero_linha)
);
CREATE INDEX ix_retorno_nosso_numero ON retorno_detalhe (nosso_numero);

CREATE TABLE outbox (
```

```

id          uuid PRIMARY KEY,
tipo        text      NOT NULL,
chave       text      NOT NULL,
payload     jsonb      NOT NULL,
criado_em   timestampz NOT NULL DEFAULT now(),
publicado_em timestampz NULL
);
CREATE INDEX ix_outbox_pendentes ON outbox (criado_em) WHERE publicado_em IS
NULL;

```

Repare na correspondência de tipos que o `COPY` binário exige sem tolerância: `int` é `i32`, `bigint` é `i64`, `smallint` é `i16`, `uuid` é `uuid::Uuid` (feature `with-uuid-1` do `tokio-postgres`) e `date` é `chrono::NaiveDate` (feature `with-chrono-0_4`). Um `u32` passado para uma coluna `int` não é convertido sozinho: é erro.

6.3 O escritor completo

```

use std::pin::pin;
use tokio_postgres::binary_copy::BinaryCopyInWriter;
use tokio_postgres::types::Type;

const CRIAR_TEMPORARIA: &str = "
    CREATE TEMP TABLE IF NOT EXISTS tmp_retorno (
        arquivo_id uuid, numero_linha int, nosso_numero bigint,
        codigo_ocorrencia smallint, data_ocorrencia date, valor_pago_centavos
        bigint
    ) ON COMMIT DELETE ROWS";

const COPY: &str = "
    COPY tmp_retorno (arquivo_id, numero_linha, nosso_numero,
        codigo_ocorrencia, data_ocorrencia,
        valor_pago_centavos)
    FROM STDIN (FORMAT BINARY)";

const TIPOS: &[Type] = &[Type::UUID, Type::INT4, Type::INT8, Type::INT2,
    Type::DATE, Type::INT8];

const CONSOLIDAR: &str = "
    WITH inseridos AS (
        INSERT INTO retorno_detalhe AS d
            (arquivo_id, numero_linha, nosso_numero, codigo_ocorrencia,
            data_ocorrencia, valor_pago_centavos)
        SELECT arquivo_id, numero_linha, nosso_numero, codigo_ocorrencia,
            data_ocorrencia, valor_pago_centavos
        FROM tmp_retorno
    )

```

```

        ON CONFLICT (arquivo_id, numero_linha) DO NOTHING
        RETURNING d.id, d.nosso_numero, d.codigo_ocorrencia,
d.valor_pago_centavos
    )
    INSERT INTO outbox (id, tipo, chave, payload)
    SELECT gen_random_uuid(), 'retorno.ocorrencia', i.nosso_numero::text,
        jsonb_build_object('detalheId', i.id, 'nossoNumero',
i.nosso_numero,
                                'ocorrencia', i.codigo_ocorrencia,
'valorPagoCentavos', i.valor_pago_centavos)
    FROM inseridos i";

#[derive(Debug, thiserror::Error)]
pub enum ErroDeEscrita {
    #[error("sem conexão do pool: {0}")]
    Pool(#[from] deadpool_postgres::PoolError),
    #[error("erro do banco: {0}")]
    Banco(#[from] tokio_postgres::Error),
}

pub struct EscritorPostgres {
    pool: deadpool_postgres::Pool,
}

impl Escritor for EscritorPostgres {
    async fn gravar(&self, lote: &Lote) -> Result<(), ErroDeEscrita> {
        let mut cliente = self.pool.get().await?; // a conexão volta ao
pool quando 'cliente' sai de escopo
        let tx = cliente.transaction().await?; // sem commit explícito,
o Drop faz rollback
        tx.batch_execute(CRIAR_TEMPORARIA).await?;

        let destino = tx.copy_in(COPY).await?;
        let mut copia = pin!(BinaryCopyInWriter::new(destino, TIPOS));
        for r in &lote.itens {
            let numero_linha = r.numero_linha as i32;
            let ocorrencia = i16::from(r.codigo_ocorrencia);
            copia.as_mut().write(&[
                &lote.arquivo_id, &numero_linha, &r.nosso_numero,
                &ocorrencia, &r.data_ocorrencia, &r.valor_pago_centavos,
            ]).await?;
        }
        copia.as_mut().finish().await?;

        tx.batch_execute(CONSOLIDAR).await?;
    }
}

```

```

        tx.commit().await?;
        Ok(())
    }
}

```

Por que esse desenho, ponto a ponto:

- **Rollback por Drop**. Uma `Transaction` do tokio-postgres que sai de escopo sem `commit` faz rollback. Qualquer `?` acima abandona a transação com segurança, sem um `defer` ou `finally` para lembrar. É o mesmo papel do `pgx.BeginFunc` do Go, garantido pelo tipo.
- **A conexão também volta por Drop**. O objeto devolvido por `pool.get()` devolve a conexão ao pool quando é solto, inclusive em erro ou `panic`.
- **Idempotência automática**. `ON CONFLICT DO NOTHING` ignora linhas já gravadas e o `RETURNING` só devolve as novas, então regravar o lote não gera eventos duplicados na outbox.
- **Tabela temporária por conexão** (`IF NOT EXISTS ... ON COMMIT DELETE ROWS`), criada uma vez em cada conexão reaproveitada pelo pool e esvaziada a cada commit.
- **Sem alocação por linha**. Cada `write` recebe um array de referências na pilha; o `BinaryCopyInWriter` codifica os valores direto no buffer do `COPY`. Diferente do `[]any` do `pgx`, não há *boxing* de valores.
- **pin!**. O escritor do `COPY` precisa ficar fixo na memória enquanto é usado (ele é um `Sink` assíncrono). A macro `std::pin::pin!` faz isso na pilha, sem alocação.

6.4 ON CONFLICT ou MERGE

A regra é a mesma dos outros cursos: `ON CONFLICT DO NOTHING` para dados imutáveis, `DO UPDATE ... WHERE` para estado que muda, deduplicação com `DISTINCT ON` na staging quando o lote pode ter duas linhas para a mesma chave, e `MERGE` só quando a lógica for rica e as chaves de transações concorrentes não se sobrepuserem.

6.5 Alternativa sem COPY: unnest

Quando o `COPY` não for viável (um pooler como o PgBouncer em modo transação com restrições), passe colunas como arrays:

```
let (linhas, nossos): (Vec<i32>, Vec<i64>) =
    lote.itens.iter().map(|r| (r.numero_linha as i32,
    r.nosso_numero)).unzip();
// ... demais colunas do mesmo jeito
tx.execute(
    "INSERT INTO retorno_detalhe (arquivo_id, numero_linha, nosso_numero,
    codigo_ocorrencia, data_ocorrencia, valor_pago_centavos)
    SELECT $1, * FROM unnest($2::int[], $3::bigint[], $4::smallint[],
    $5::date[], $6::bigint[])
    ON CONFLICT (arquivo_id, numero_linha) DO NOTHING",
    [&lote.arquivo_id, &linhas, &nossos, &ocorrencias, &datas, &valores],
).await?;
```

Aqui há alocações (um `Vec` por coluna), mas por lote, não por linha.

6.6 E o sqlx?

O `sqlx` é a biblioteca de banco mais popular em Rust, com uma vantagem notável: as macros `sqlx::query!` verificam cada consulta contra o schema real em tempo de compilação (ou contra um cache gerado por `cargo sqlx prepare`, para compilar sem banco no CI), o equivalente ao `sqlc` do Go. Para o `COPY`, porém, ele oferece uma interface de bytes crus, sem o codificador binário tipado do `tokio-postgres`. Duas combinações razoáveis:

- **Só `tokio-postgres`** (este curso): um driver, um pool, `COPY` binário tipado; consultas de metadados escritas à mão e cobertas por testes de integração.
- **`sqlx` para as consultas e `COPY` em formato CSV** pelo `copy_in_raw`: consultas verificadas pelo compilador, ao custo de formatar o CSV do lote (perde um pouco de desempenho e exige cuidado com escape).

O que não é razoável é manter dois drivers e dois pools no mesmo worker só para ter as duas coisas: o orçamento de conexões (Módulo 10) dobra sem nenhum ganho.

6.7 Ajustes do lado do banco

Os mesmos dos outros cursos: lotes de 1.000 a 10.000 linhas, medidos; uma transação por lote, nunca por arquivo; `synchronous_commit = off` só com reconciliação e dados reconstruíveis; autovacuum ajustado nas tabelas grandes de inserção; e carga inicial massiva sem índices secundários, nunca com o sistema em produção.

Exercício do Módulo 6

Crie as tabelas e implemente três escritores: `execute` por linha dentro de uma transação, `unnest` e `COPY` binário + consolidação. Grave 1 milhão de registros com cada um, em lotes de 5.000 e 4 escritores, e anote linhas por segundo. Regrave o mesmo arquivo e confirme que `retorno_detalhe` e `outbox` não mudaram. Passe de propósito um `u32` no lugar do `i32` do número da linha e leia o erro do `COPY` binário. Se os workers em .NET e Go estiverem prontos, compare as três vazões: com o banco como gargalo, elas devem ficar próximas, e a diferença de Rust deve aparecer na memória e na CPU do worker.

Módulo 7 — Leitura em massa

As regras são as mesmas em qualquer linguagem: só as colunas necessárias, linhas consumidas em streaming, paginação por chave em vez de `OFFSET` e um índice que transforme a consulta numa varredura curta. Em Rust, o streaming é um `Stream` assíncrono de linhas, e o `COPY TO` exporta milhões de linhas como blocos de bytes sem decodificar nenhuma. Um detalhe específico de Rust muda o jeito de pensar em prazos: cancelar uma future **não** cancela a consulta no servidor.

7.1 Streaming de linhas

`client.query(...)` devolve um `Vec<Row>` com o resultado inteiro, ótimo para resultados pequenos. Para milhões de linhas, `query_raw` devolve um `RowStream`, que entrega cada linha conforme ela chega da rede:

```

use futures::TryStreamExt;
use std::pin::pin;

let cliente = pool.get().await?;
let linhas = cliente
    .query_raw(
        "SELECT id, nosso_numero, valor_pago_centavos FROM retorno_detalhe
WHERE data_ocorrendia = $1",
        [dia], // NaiveDate
    )
    .await?;
let mut linhas = pin!(linhas);

while let Some(linha) = linhas.try_next().await? {
    let id: i64 = linha.get(0);
    let nosso_numero: i64 = linha.get(1);
    let valor: i64 = linha.get(2);
    // processa uma linha por vez, memória constante
}

```

Consoma o stream no mesmo escopo em que a conexão (`cliente`) vive: é ela que mantém a consulta aberta. E o aviso de sempre: enquanto o laço roda, um snapshot do MVCC fica preso e impede o autovacuum de limpar versões antigas no banco inteiro. Para jobs longos, prefira páginas curtas.

7.2 Keyset em vez de OFFSET

`OFFSET 1000000 LIMIT 1000` lê e descarta um milhão de linhas para devolver mil. Keyset parte da última chave vista:

```

let consulta = cliente
    .prepare("SELECT id, nosso_numero, valor_pago_centavos FROM
retorno_detalhe
        WHERE data_ocorrendia = $1 AND id > $2 ORDER BY id LIMIT 5000")
    .await?; // preparada uma vez, reaproveitada a cada página

let mut ultimo: i64 = 0;
loop {
    let pagina = cliente.query(&consulta, [&dia, &ultimo]).await?;
    let Some(fim) = pagina.last() else { break };
    ultimo = fim.get(0); // a próxima página começa depois da última chave
    processar(&pagina).await?;
}

```

Com um índice em `(data_ocorrencia, id)`, a milésima página custa o mesmo que a primeira. Para ordenações por mais de uma coluna, `WHERE (data_ocorrencia, id) > ($1, $2)`. E keyset paraleliza bem: faixas de IDs diferentes para tarefas diferentes de um `JoinSet`, com um semáforo limitando quantas rodam juntas.

7.3 Exportações com COPY TO

`copy_out` devolve o CSV gerado pelo PostgreSQL como um stream de blocos de bytes, que você escreve direto num arquivo, num upload ou numa resposta HTTP:

```
let sql = format!(
    "COPY (SELECT nosso_numero, codigo_ocorrencia, valor_pago_centavos
        FROM retorno_detalhe WHERE data_ocorrencia = '{}')
    TO STDOUT (FORMAT csv, HEADER, DELIMITER ';')",
    dia.format("%Y-%m-%d"), // valor tipado, formatado por nós: nunca texto
    vindo de fora
);
let blocos = cliente.copy_out(sql.as_str()).await?;
let mut blocos = pin!(blocos);
while let Some(bloco) = blocos.try_next().await? {
    destino.write_all(&bloco).await?;
}
destino.flush().await?;
```

Nenhuma linha vira struct Rust, então o custo de CPU do worker fica perto de zero. O `COPY` não aceita parâmetros: só monte o comando com valores tipados que você mesmo formatou (uma `NaiveDate` é segura); para filtros variáveis vindos de fora, crie uma função no banco e chame-a dentro do `COPY`.

7.4 Escolhendo a ferramenta de acesso

Ferramenta	Melhor para	Observações
<code>tokio-postgres</code>	Caminho quente, <code>COPY</code> binário, controle total	Linhas lidas por posição ou nome, sem verificação em compilação
<code>sqlx</code>	Consultas fixas verificadas contra o schema no build	<code>query_as!</code> mapeia para structs; <code>fetch</code> devolve um stream; <code>COPY</code> só em bytes crus
Diesel (com <code>diesel-async</code>)	Consultas montadas por um DSL tipado	Muito seguro, curva de aprendizado própria; raro em workers de volume

Ferramenta	Melhor para	Observações
SeaORM	Modelos com relações e telas CRUD	Mais abstração, menos controle do SQL

Para este worker, `tokio-postgres` em tudo mantém um driver e um pool. Se o projeto crescer em consultas de metadados, migrar essas consultas para `sqlx` (com o `COPY` em CSV, como discutido no Módulo 6.6) troca um pouco de desempenho por verificação em compilação.

7.5 Índices e particionamento

As regras de índices são do PostgreSQL e estão detalhadas no curso de .NET (Módulo 7.5): igualdade antes de intervalo em índices compostos, `INCLUDE` para *index-only scans*, índices parciais para subconjuntos pequenos, BRIN para colunas que crescem com a ordem física, e `EXPLAIN (ANALYZE, BUFFERS)` para confirmar.

O particionamento por mês da tabela de detalhes também é o mesmo:

```
CREATE TABLE retorno_detalhe (
    id                bigint GENERATED ALWAYS AS IDENTITY,
    data_ocorrencia   date    NOT NULL,
    arquivo_id        uuid    NOT NULL,
    numero_linha      int     NOT NULL,
    nosso_numero      bigint NOT NULL,
    valor_pago_centavos bigint NOT NULL,
    PRIMARY KEY (id, data_ocorrencia),
    UNIQUE (arquivo_id, numero_linha, data_ocorrencia)
) PARTITION BY RANGE (data_ocorrencia);

CREATE TABLE retorno_detalhe_2026_10 PARTITION OF retorno_detalhe
FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
```

Com ele, o `ON CONFLICT` do Módulo 6 passa a ser `(arquivo_id, numero_linha, data_ocorrencia)`, e a retenção vira `DETACH + DROP` de partições antigas.

7.6 Prazos: cancelar a future não cancela a consulta

Em Rust é natural limitar uma operação com

`tokio::time::timeout(Duration::from_secs(30), cliente.query(...))`. Quando o prazo estoura, a future é descartada e o seu código segue em frente, mas **o PostgreSQL continua executando a consulta**: descartar a future só para de esperar pela resposta. Duas defesas, de preferência juntas:

- **statement_timeout na conexão**, configurado no pool (`config.options = Some("-c statement_timeout=60s".into())`): o próprio servidor aborta consultas longas, mesmo que o código esqueça o prazo.
- **Cancelamento explícito**: guarde `cliente.cancel_token()` antes da consulta e, quando o prazo estourar, chame `cancel_query` com ele. É o que o `pgx` e o `Npgsql` fazem por baixo quando o contexto ou o token são cancelados.

Leituras analíticas pesadas podem ir para uma réplica com um segundo pool, lembrando do atraso de replicação: o que precisa ler o que acabou de escrever (o fan-in dos chunks) lê do primário.

Exercício do Módulo 7

Com alguns milhões de linhas carregadas, compare a página 1 e a página 500 com `OFFSET` e com `keyset` usando `EXPLAIN (ANALYZE, BUFFERS)`. Implemente a leitura em streaming com `query_raw` e confirme pela memória do processo que ela não cresce com o tamanho do resultado. Exporte um dia inteiro com `copy_out` e compare tempo e CPU com a mesma exportação feita linha a linha. Por fim, envolva um `SELECT pg_sleep(30)` num `tokio::time::timeout` de 2 segundos e confirme em `pg_stat_activity` que a consulta continua rodando; depois repita com `statement_timeout` e veja o servidor abortá-la.

Módulo 8 — Confiabilidade: quando algo quebra no meio

O desenho é o dos outros cursos: entrega pelo menos uma vez (fila, retries, lease) mais idempotência em cada camada (chaves naturais, `ON CONFLICT`, outbox). Em Rust, três peças ganham uma forma própria: o lease como uma tarefa cujo resultado diz se a posse foi perdida, o retry como uma função genérica sobre um trait de "erro transitório" e a reconciliação sobre um contexto que o leitor devolve por valor.

8.1 O mapa de falhas

Falha	Exemplo	Resposta do design
Mesmo arquivo enviado duas vezes	Banco reenvia o retorno do dia	<code>UNIQUE (cliente_id, sha256)</code> na ingestão
Worker morre no meio	OOM, deploy, nó removido	Fila reentrega, lease expira, escrita idempotente
Erro transitório	Timeout, failover, deadlock	Retry com backoff e jitter
Erro permanente	Layout inválido	Sem retry: <code>REJEITADO</code>

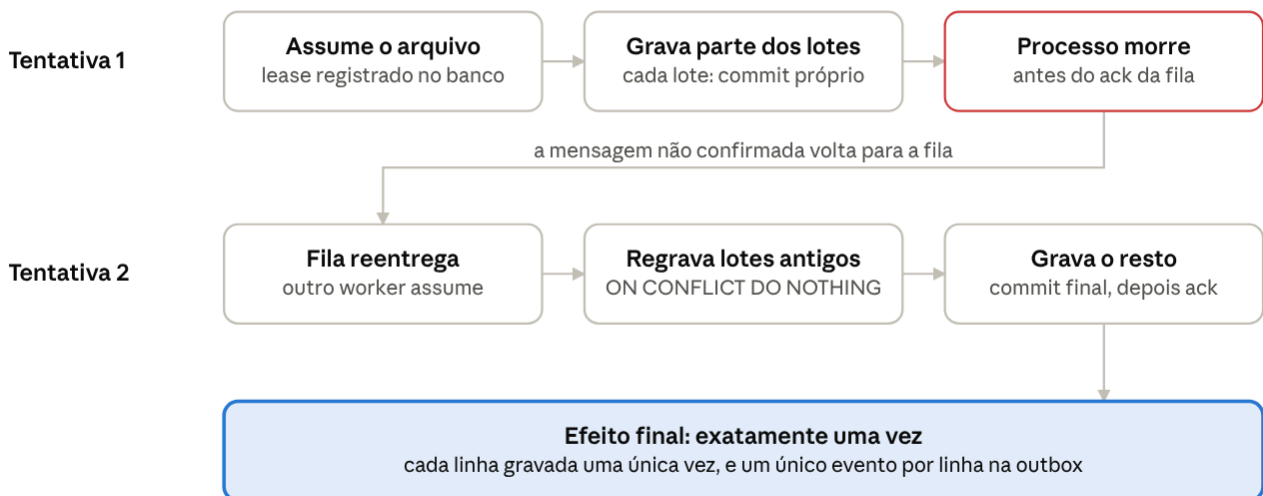
Falha	Exemplo	Resposta do design
Gravou, não publicou	Broker fora após o commit	Outbox transacional
Evento publicado duas vezes	Relay cai entre publicar e marcar	Consumidor deduplica pelo ID
Dado incompleto silencioso	Bug perde linhas sem erro	Reconciliação com o trailer

8.2 Idempotência camada por camada

Ingestão (hash único por cliente), tomada do arquivo (`UPDATE` condicional), escrita (`ON CONFLICT DO NOTHING` em (`arquivo_id`, `numero_linha`)), eventos (a outbox só recebe eventos de linhas realmente inseridas) e consumidores (tabela `inbox`). O teste de fogo: "posso reprocessar este arquivo do zero agora, em produção, sem medo?".

Pelo menos uma vez + idempotência = efeito de exatamente uma vez

Um arquivo processado duas vezes por causa de uma queda · em vermelho, a falha; em destaque, o resultado



um arquivo, duas tentativas · efeito único

8.3 Lease: uma tarefa que vigia a posse

O worker assume o arquivo com o `UPDATE` condicional dos outros cursos (`SET status = 'PROCESSANDO'`, `processando_ate = now() + interval '5 minutes'`, `worker_id = $2`, `tentativas = tentativas + 1 WHERE id = $1 AND (status = 'RECEBIDO' OR (status = 'PROCESSANDO' AND processando_ate < now())) RETURNING tentativas`).

Durante o processamento, uma tarefa separada renova o lease a cada minuto. Se a renovação falhar, ela cancela o token do arquivo e termina com erro, e esse erro é a causa que o worker devolve:

```
pub async fn com_lease<F, Fut, T>(
    &self,
    msg: &Mensagem,
    pai: &CancellationToken,
    trabalho: F,
) -> Result<T, ErroDeProcessamento>
where
    F: FnOnce(CancellationToken) -> Fut,
    Fut: Future<Output = Result<T, ErroDeProcessamento>>,
{
    let token = pai.child_token();

    let renovacao = {
        let (arquivos, fila, msg, token) =
            (self.arquivos.clone(), self.fila.clone(), msg.clone(),
token.clone());
        tokio::spawn(async move {
            let mut intervalo =
tokio::time::interval(Duration::from_secs(60));
            intervalo.tick().await; // o primeiro tick é imediato
            loop {
                tokio::select! {
                    _ = token.cancelled() => return Ok(()), // fim normal: o
trabalho terminou
                    _ = intervalo.tick() => {}
                }
                if let Err(e) = arquivos.renovar_lease(msg.arquivo_id).await
{
                    token.cancel(); // perdeu a posse: interrompe o pipeline
                    return Err(e);
                }
                let _ = fila.estender_visibilidade(&msg,
Duration::from_secs(300)).await;
            }
        })
    };

    let resultado = trabalho(token.clone()).await;
    token.cancel(); // encerra a renovação em qualquer caminho
    match renovacao.await {
        Ok(Err(e)) => Err(ErroDeProcessamento::LeasePerdido(e)), // a causa
```

```

    real, não "cancelado"
        _ => resultado,
    }
}

```

O que no Go era `context.WithCancelCause`, aqui é o resultado da tarefa de renovação: se ela terminou com erro, o lease foi perdido, e isso tem precedência sobre o "cancelado" que o pipeline devolveu por consequência. A tarefa nunca vaza, porque o `token.cancel()` depois do trabalho a encerra em todos os caminhos, e ela é aguardada com `.await` antes de a função terminar.

8.4 Retry com backoff e jitter

Uma função genérica, um trait que diz quais erros valem a pena repetir e um *decorator* para o escritor:

```

pub trait Transitorio {
    fn transitorio(&self) -> bool;
}

pub async fn com_retry<T, E, F, Fut>(tentativas: u32, base: Duration, mut f:
F) -> Result<T, E>
where
    F: FnMut() -> Fut,
    Fut: Future<Output = Result<T, E>>,
    E: Transitorio,
{
    let mut i = 0;
    loop {
        match f().await {
            Ok(v) => return Ok(v),
            Err(e) if i + 1 < tentativas && e.transitorio() => {
                let teto = (base * 2u32.pow(i)).as_millis() as u64;
                let espera =
Duration::from_millis(rand::random_range(0..=teto)); // full jitter
                tokio::time::sleep(espera).await;
                i += 1;
            }
            Err(e) => return Err(e),
        }
    }
}

impl Transitorio for ErroDeEscrita {
    fn transitorio(&self) -> bool {
        use tokio_postgres::error::SqlState;

```

```

        match self {
            ErroDeEscrita::Pool(_) => true, // timeout do pool ou falha ao
abrir conexão
            ErroDeEscrita::Banco(e) => {
                e.is_closed() // a conexão caiu
                || e.code().is_some_and(|c| {
                    *c == SqlState::T_R_SERIALIZATION_FAILURE
                    || *c == SqlState::T_R_DEADLOCK_DETECTED
                    || *c == SqlState::ADMIN_SHUTDOWN
                    || *c == SqlState::TOO_MANY_CONNECTIONS
                })
            }
        }
    }
}

pub struct EscritorComRetry<E>(pub E);

impl<E: Escritor> Escritor for EscritorComRetry<E> {
    async fn gravar(&self, lote: &Lote) -> Result<(), ErroDeEscrita> {
        com_retry(5, Duration::from_millis(200), ||
self.0.gravar(lote)).await
    }
}

```

O compilador ajuda aqui de um jeito que os outros cursos não tinham: `match self` sobre `ErroDeEscrita` precisa cobrir todas as variantes. Se alguém acrescentar uma nova fonte de erro ao escritor, o código de retry deixa de compilar até que se decida se ela é transitória. As razões do desenho são as mesmas: backoff exponencial para dar tempo a um failover, jitter para que as réplicas não tentem juntas, só erros transitórios e, acima de tudo, a garantia de que `gravar` é uma transação idempotente, inclusive quando a conexão cai durante o commit e não se sabe se ele aconteceu.

8.5 DLQ e confirmação

As mesmas regras: DLQ após N recebimentos, o contador `tentativas` da tabela `arquivo` como segunda barreira e a regra de ouro, **apagar a mensagem só depois do commit final**. A DLQ precisa de dono, alarme e uma ação de reenvio.

8.6 O relay da outbox

```

pub async fn rodar_relay(pool: Pool, publicador: impl Publicador,
cancelamento: CancellationToken) {
    loop {
        let publicados = match publicar_lote(&pool, &publicador).await {

```

```

        Ok(n) => n,
        Err(e) => {
            tracing::error!(erro = %e, "relay da outbox");
            0
        }
    };
    if publicados > 0 {
        continue; // havia eventos: tenta o próximo lote já
    }
    tokio::select! {
        _ = cancelamento.cancelled() => return,
        _ = tokio::time::sleep(Duration::from_millis(500)) => {}
    }
}

}

async fn publicar_lote(pool: &Pool, publicador: &impl Publicador) ->
Result<usize, ErroDoRelay> {
    let mut cliente = pool.get().await?;
    let tx = cliente.transaction().await?;
    let linhas = tx.query(
        "SELECT id, tipo, chave, payload::text FROM outbox
        WHERE publicado_em IS NULL ORDER BY criado_em LIMIT 200 FOR UPDATE
        SKIP LOCKED", &[]).await?;
    if linhas.is_empty() {
        return Ok(0); // o Drop da transação faz rollback: nada a desfazer
    }
    let eventos: Vec<Evento> = linhas.iter().map(Evento::da_linha).collect();
    publicador.publicar(&eventos).await?; // o ID do evento vai como ID da
    mensagem
    let ids: Vec<Uuid> = eventos.iter().map(|e| e.id).collect();
    tx.execute("UPDATE outbox SET publicado_em = now() WHERE id = ANY($1)", &
[&ids]).await?;
    tx.commit().await?;
    Ok(eventos.len())
}

```

O relay publica **pelo menos uma vez** (se cair entre publicar e o commit, o lote volta), então os consumidores deduplicam pelo ID. Ele lê e escreve a mesma outbox que os workers em .NET e Go: um relay, em qualquer das três linguagens, atende todos.

8.7 O contexto do arquivo e a reconciliação

O `ContextoDoArquivo` que o leitor recebe e devolve no Módulo 4:

```

#[derive(Debug, Default)]
pub struct ContextoDoArquivo {
    limite_de_erros: usize,
    pub linhas_lidas: u32,
    pub linhas_rejeitadas: u32,
    pub trailer: Option<(u32, i64)>, // None = arquivo sem trailer
    pub primeiros_erros: Vec<(u32, ErroDeLinha)>,
}

impl ContextoDoArquivo {
    pub fn novo(limite_de_erros: usize) -> Self {
        Self { limite_de_erros, ..Default::default() }
    }
    pub fn registrar_erro(&mut self, linha: u32, erro: ErroDeLinha) {
        self.linhas_rejeitadas += 1;
        if self.primeiros_erros.len() < self.limite_de_erros {
            self.primeiros_erros.push((linha, erro));
        }
    }
    pub fn registrar_trailer(&mut self, quantidade: u32, total: i64) {
        self.trailer = Some((quantidade, total));
    }
    pub fn registrar_fim(&mut self, linhas: u32) {
        self.linhas_lidas = linhas;
    }
}

```

Repare que ele não tem `Mutex` nem `Arc`: só o leitor o modifica, e ele só chega ao worker depois que o leitor terminou e o devolveu. Isso não é convenção, é consequência dos tipos: não há como outra tarefa ter acesso a ele enquanto o leitor o possui. As regras de reconciliação são as dos outros cursos:

1. `trailer` igual a `None`: arquivo truncado, `FALHOU`.
2. Quantidade: registros gravados + `linhas_rejeitadas` deve bater com o trailer, ajustada ao manual de cada banco.
3. Valor: a soma gravada deve bater com o total; com rejeições, o arquivo vai para revisão com o relatório de `primeiros_erros`.

Um `enum` modela o resultado e obriga quem chama a tratar cada caso:

```
pub enum Reconciliacao {
    Confere,
    SemTrailer,
    QuantidadeDiverge { gravados: i64, rejeitados: u32, trailer: u32 },
    ValorDiverge { gravado: i64, trailer: i64 },
    ComRejeicoes { rejeitados: u32 },
}
```

8.8 Desligamento gracioso

No SIGTERM, o token do processo é cancelado. O laço de consumo para de buscar mensagens; os escritores terminam o lote em andamento e param antes do próximo (Módulo 5.3); uma transação interrompida faz rollback pelo `Drop`; nenhuma mensagem de arquivo incompleto é confirmada; e o `TaskTracker` espera as tarefas em andamento antes de o `main` retornar. A fila reentrega, o lease expira, outro worker assume e a idempotência cuida do resto.

Exercício do Módulo 8

O teste de caos dos outros cursos: processe um arquivo de 2 milhões de linhas com duas réplicas e mate uma delas com `kill -9`, três vezes, em momentos diferentes; confira que `retorno_detalhe` bate com o trailer, que não há eventos duplicados e que o arquivo terminou `CONCLUIDO`. Derrube o PostgreSQL por 30 segundos e confirme que `com_retry` absorve a queda. Acrescente uma variante nova a `ErroDeEscrita` e veja o `impl Transitorio` parar de compilar. Se os workers em .NET e Go estiverem prontos, rode uma réplica de cada linguagem na mesma fila e mate qualquer uma.

Módulo 9 — Observabilidade e medição de performance

As perguntas são as mesmas: vazão, atraso, erros e saturação. Em Rust, o centro da observabilidade é o crate `tracing`, que unifica logs e spans num só modelo e se liga ao OpenTelemetry. Para desempenho, não há um `pprof` embutido no runtime como no Go: as ferramentas são externas (`perf`, `samply`, `cargo flamegraph`), mais o `tokio-console` para enxergar as tarefas e o `criterion` para benchmarks.

9.1 O que medir

Métrica	Tipo	Pergunta que responde
<code>retorno.arquivos.processados</code> (por status e banco)	Counter	Quantos terminaram, e como?

Métrica	Tipo	Pergunta que responde
<code>retorno.linhas.gravadas</code>	Counter	Qual a vazão real?
<code>retorno.arquivo.duracao</code>	Histogram (s)	Estamos dentro do SLA? Qual o p95 e o p99?
<code>retorno.lote.duracao</code>	Histogram (ms)	O banco está ficando lento?
Idade da mensagem mais antiga na fila	Gauge (s)	Estamos atrasados?
Ocupação do canal de lotes (<code>tx.len()</code>)	Gauge	O gargalo é a leitura ou a escrita?
Conexões em uso no pool (<code>pool.status()</code>)	Gauge	O pool ou o banco saturou?
Tarefas vivas e fila global do runtime	Runtime	Alguma tarefa vazando? O runtime está sobrecarregado?

Não há métricas de GC porque não há GC. Em compensação, o número de tarefas vivas do tokio (`Handle::current().metrics().num_alive_tasks()`) é o sinal equivalente ao número de goroutines no Go: crescendo sem parar ao longo de horas, é vazamento.

9.2 tracing: logs e spans no mesmo modelo

Um *span* é um intervalo com atributos ("processando o arquivo X"); um *evento* é um log dentro dele, que herda os atributos do span. A macro `#[instrument]` cria um span por chamada de função:

```
#[tracing::instrument(
    name = "processar-arquivo",
    skip(self, cancelamento),
    fields(arquivo_id = %msg.arquivo_id, banco = %msg.banco),
)]
async fn processar_mensagem(&self, msg: Mensagem, cancelamento:
&CancellationToken) {
    // ...
    tracing::info!(linhas = contexto.linhas_lidas, rejeitadas =
contexto.linhas_rejeitadas, "arquivo concluído");
}
```

O `info!` acima sai no log com `arquivo_id` e `banco` sem repeti-los, porque está dentro do span. As regras dos outros cursos valem: um span por arquivo e um por lote, nunca por linha; erros de linha agregados no relatório do arquivo. A saída em JSON vem do `tracing-subscriber`:

```
tracing_subscriber::registry()
    .with(tracing_subscriber::EnvFilter::from_default_env()) //
RUST_LOG=info,cobranca=debug
    .with(tracing_subscriber::fmt::layer().json())
    .with(tracing_opentelemetry::layer().with_tracer(tracer)) // os mesmos
spans viram traces OTLP
    .init();
```

A terceira camada é a ponte: cada span do `tracing` vira um span do OpenTelemetry, com o mesmo `trace_id` que aparece nas linhas de log. Para continuar o trace iniciado na ingestão, extraia o contexto dos atributos da mensagem com o propagador `TraceContextPropagator` (um `HashMap<String, String>` já serve como portador) e defina-o como pai do span com `set_parent`, do trait `OpenTelemetrySpanExt`.

9.3 Métricas

Com o provedor do OpenTelemetry configurado (exportador OTLP, leitor periódico), os instrumentos são criados uma vez e guardados no worker:

```
pub struct Metricas {
    linhas: Counter<u64>,
    duracao_lote: Histogram<f64>,
}

impl Metricas {
    pub fn novas() -> Self {
        let m = opentelemetry::global::meter("cobranca.retorno");
        Self {
            linhas: m.u64_counter("retorno.linhas.gravadas").with_unit("{linha}").build(),
            duracao_lote:
m.f64_histogram("retorno.lote.duracao").with_unit("ms").build(),
        }
    }

    pub fn lote_gravado(&self, linhas: usize, duracao: Duration) {
        self.linhas.add(linhas as u64, &[]);
        self.duracao_lote.record(duracao.as_secs_f64() * 1000.0, &[]);
    }
}
```

A API do OpenTelemetry para Rust ainda muda entre versões (o método final dos builders já se chamou `init`, por exemplo); confira a documentação da versão que você fixar no `Cargo.toml`. Gauges observáveis leem `tx.len()` do canal e `pool.status()` a cada

coleta, sem custo no caminho quente. A regra de cardinalidade é a de sempre: `status` e `banco` como atributos, nunca `arquivo_id`.

9.4 Profiling

Ferramenta	Pergunta que responde	Uso típico
<code>samply</code> ou <code>cargo flamegraph</code>	Onde a CPU está sendo gasta?	Gravar 30 s sob carga e abrir o flame graph
<code>perf</code>	O mesmo, direto no Linux e em containers	Quando não dá para instalar mais nada
<code>pprof-rs</code>	Perfil de CPU servido pelo próprio processo	Um endpoint de administração, como o <code>pprof</code> do Go
<code>dhat</code> ou <code>heaptrack</code>	Quem aloca, quanto e onde?	Verificar o "zero alocação por linha" no programa inteiro
<code>tokio-console</code>	Quais tarefas existem, quanto tempo ficam sem ceder a thread?	Tarefas lentas entre dois <code>.await</code> , vazamentos de tarefa

O `tokio-console` é o que mais se aproxima de uma visão de runtime: ele mostra cada tarefa viva, quanto tempo ela ficou ocupada, quantas vezes foi acordada e alerta sobre tarefas que seguram a thread por tempo demais (o problema do Módulo 3.7). Ele exige compilar com `--cfg tokio_unstable` e o crate `console-subscriber`, então costuma ficar num perfil de build de diagnóstico, não no binário de produção.

9.5 Benchmarks

O `criterion` faz o papel do BenchmarkDotNet e do `testing.B`: aquecimento, repetições e análise estatística, comparando cada execução com a anterior:

```
fn bench_parse(c: &mut Criterion) {
    let linhas = gerar_linhas(10_000);
    c.bench_function("parse_detalhe 10k", |b| {
        b.iter(|| {
            for (i, l) in linhas.iter().enumerate() {
                black_box(cnaab::parse_detalhe(black_box(l), i as
u32).unwrap());
            }
        })
    });
}

criterion_group!(benches, bench_parse);
criterion_main!(benches);
```

`std::hint::black_box` impede o compilador de eliminar o trabalho que está sendo medido, um risco real com o otimizador do Rust. Rode com `cargo bench`; o `criterion` informa se a mudança foi estatisticamente significativa em relação à última execução, e `-save-baseline` guarda uma referência nomeada para comparar versões.

9.6 O perfil de compilação de produção

O `cargo build --release` padrão já é otimizado, e alguns ajustes no `Cargo.toml` valem para um worker de longa duração:

```
[profile.release]
lto = "thin"                # otimização entre crates; "fat" ganha um pouco
                             # mais, compila bem mais devagar
codegen-units = 1           # mais otimização, compilação mais lenta
debug = "line-tables-only"  # nomes e linhas nos flame graphs, sem inchar o
                             # binário
# panic = "abort" NÃO: o JoinSet do Módulo 5 depende de panics que só
                             # encerram a tarefa
```

O comentário sobre `panic` é o ponto mais importante: muitos guias recomendam `panic = "abort"` para binários menores, mas com ele um `panic` num escritor derrubaria o processo inteiro, como no Go, e a garantia do Módulo 5.1 desapareceria.

9.7 Teste de carga e alertas

O mesmo método dos outros cursos (ambiente completo, distribuição real de tamanhos, carga em degraus) e os mesmos alertas de sintoma (arquivo mais antigo na fila, DLQ, divergência de reconciliação, silêncio numa janela de pico), mais o equivalente Rust do alerta de goroutines: tarefas vivas do tokio crescendo sem parar.

Exercício do Módulo 9

Instrumente o worker com `#[instrument]` por arquivo e por lote, a saída JSON e a ponte para o OpenTelemetry, e confirme no Aspire Dashboard que logs e traces compartilham o `trace_id`. Rode o teste de carga e encontre o gargalo pelas métricas de ocupação do canal e de conexões em uso. Grave um flame graph com `samply` sob carga. Compile uma vez com `tokio_unstable` e o `console-subscriber`, introduza um `std::thread::sleep` de 200 ms num escritor e veja o `tokio-console` apontar a tarefa culpada.

Módulo 10 — Escala horizontal e deploy

Workers sem estado escalam com réplicas, o sinal de escala é o atraso da fila e o teto é o banco, como nos outros cursos. O que Rust acrescenta ao deploy é o footprint: um binário estático de poucos MB, sem runtime para aquecer e com memória que acompanha o que está vivo. Isso permite mais réplicas por nó, mas não muda o teto, que continua sendo o orçamento de conexões do PostgreSQL.

10.1 Escalar pela fila

O `ScaledObject` do KEDA é o mesmo dos outros cursos, apontando para o Deployment em Rust:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: retorno-worker-rust
spec:
  scaleTargetRef:
    name: retorno-worker-rust
  minReplicaCount: 1
  maxReplicaCount: 8           # teto calculado pelo banco (10.2)
  pollingInterval: 15
  cooldownPeriod: 300
  triggers:
    - type: aws-sqs-queue
      authenticationRef:
        name: keda-aws-credenciais
      metadata:
        queueURL: https://sqs.sa-east-1.amazonaws.com/<conta>/retornos
        awsRegion: sa-east-1
        queueLength: "4"
```

Com subida em poucos milissegundos, escalar a zero custa basicamente o tempo de agendar o pod, o que torna `minReplicaCount: 0` mais atraente para filas com longos períodos ociosos.

10.2 O teto é o banco

$$R_{\max} = \left\lfloor \frac{C}{A \cdot E + X} \right\rfloor$$

C conexões reservadas pelo banco para o pipeline, A arquivos em paralelo por réplica (o semáforo do Módulo 5.4), E escritores por arquivo e X conexões extras (lease, metadados). Com C = 80, A = 2, E = 4 e X = 2, são 8 réplicas. No `deadpool-postgres`, o `max_size` do pool de cada réplica é exatamente $A \times E + X$, e um pedido de conexão além disso espera (com `timeouts.wait` configurado, falha com erro de pool em vez de esperar para sempre). Se workers em Rust, Go e .NET dividirem a fila durante uma migração, o C é único e precisa ser repartido entre os `ScaledObject`s.

10.3 O Deployment do worker

```
spec:
  template:
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: worker
          image: <registro>/cobranca-retorno-worker-rust:1.0.0
          resources:
            requests: { cpu: "1", memory: 128Mi }
            limits:   { cpu: "2", memory: 256Mi }
          env:
            - name: RUST_LOG
              value: "info"
          livenessProbe:
            httpGet: { path: /vivo, port: 9090 }
            periodSeconds: 30
            failureThreshold: 4
```

Decisões específicas de Rust:

- **Memória.** O worker precisa do buffer de leitura, dos lotes em voo (4 no canal + 1 por escritor, cerca de 160 KB cada) e dos buffers das conexões. Os números acima são uma ordem de grandeza plausível, não uma medida: confirme no teste de carga. Não há `GOMEMLIMIT` nem limite de heap para configurar; sem GC, a memória é a soma do que está vivo mais a fragmentação do alocador, outro motivo para usar `mimalloc` ou `jemalloc` (Módulo 3.6).

- **CPU.** O runtime do tokio cria uma thread de trabalho por núcleo disponível, e no Linux esse número respeita a cota de CPU do cgroup, ou seja, o **limite**. Como no Go, sem limite de CPU num nó grande, o worker criaria dezenas de threads. Defina o limite ou fixe as threads com a variável `TOKIO_WORKER_THREADS`, que o tokio lê na subida.
- **Liveness por progresso.** Um `AtomicU64` com o instante do último lote, atualizado pelo laço principal, e um servidor de administração mínimo (com `axum`, por exemplo) que responde 503 em `/vivo` se não houver progresso há três minutos. As regras dos outros cursos valem: tolerância folgada e nunca depender do banco.

10.4 A imagem

Com TLS em Rust puro (`rustls`, usado por padrão pelo AWS SDK e disponível para o `tokio-postgres` pelo crate `tokio-postgres-rustls`), o worker não depende de OpenSSL e pode ser compilado como binário totalmente estático com musl:

```
FROM rust:1.99 AS build
RUN rustup target add x86_64-unknown-linux-musl \
  && apt-get update && apt-get install -y --no-install-recommends musl-tools
WORKDIR /src
COPY . .
RUN cargo build --release --locked --target x86_64-unknown-linux-musl

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /src/target/x86_64-unknown-linux-musl/release/worker
/worker
USER nonroot:nonroot
ENTRYPOINT ["/worker"]
```

Três observações. Com musl, troque o alocador (Módulo 3.6): o `malloc` do musl é o maior risco de desempenho deste Dockerfile. A compilação de Rust é lenta, e copiar todo o código antes do `cargo build` recompila todas as dependências a cada mudança; ferramentas como `cargo-chef` separam a compilação das dependências numa camada de cache própria. E `--locked` garante que o build usa exatamente as versões do `Cargo.lock` comitado.

Rust também suporta PGO (otimização guiada por perfil), pelas opções `-Cprofile-generate` e `-Cprofile-use` do compilador, com o `cargo-pgo` automatizando o ciclo. O processo é mais trabalhoso que o `default.pgo` do Go (instrumentar, rodar uma carga representativa, recompilar), e para um worker dominado por I/O de banco o ganho costuma ser pequeno; vale mais em serviços limitados por CPU.

10.5 Particionamento, recuperação e custo

As mesmas estratégias dos outros cursos: filas rápida e pesada roteadas pelo tamanho do arquivo, filas por origem, prioridades de negócio, cada partição com seu `ScaledObject` e sua fatia do orçamento de conexões. Para desastres, bucket replicado, backup do banco com recuperação para um ponto no tempo e um procedimento testado de reenfileirar arquivos. No custo, o footprint menor do worker em Rust reduz a conta de computação, mas os três itens que crescem sem ninguém perceber continuam os mesmos: arquivos no storage, volume de logs e armazenamento do banco.

Exercício do Módulo 10

Num cluster local com KEDA e LocalStack, faça o deploy do worker e publique 200 arquivos de uma vez; observe as réplicas subindo e voltando. Meça o uso de memória de cada réplica durante o maior arquivo e compare com os workers em .NET e Go, se estiverem prontos. Compare o tamanho das imagens com `docker images`. Por fim, rode com e sem limite de CPU num nó com muitos núcleos e registre o número de threads de trabalho do tokio na subida.

Projeto final — o worker de retorno em Rust, passo a passo

O mesmo projeto dos cursos de .NET e Go: um worker que consome a fila, processa arquivos de qualquer tamanho com memória constante, grava de forma idempotente, publica eventos pela outbox e sobrevive a quedas em qualquer ponto. Ele usa as mesmas tabelas e as mesmas mensagens e pode dividir a fila com os workers das outras duas linguagens.

Estrutura do workspace

```
cobranca-retorno-rs/
├ Cargo.toml                workspace, [profile.release] do Módulo 9.6
├ Cargo.lock                comitado: builds reproduzíveis
├ crates/
│   └─ cnab/                RegistroRetorno, parse_detalhe,
parse_trailer (sem dependências de I/O)
│   └─ pipeline/            leitor, Lote, ContextoDoArquivo,
processar_arquivo, trait Escritor
│   └─ escrita/             EscritorPostgres, EscritorComRetry,
com_retry, Transitorio
│   └─ worker/              Worker, consumir, processar_mensagem,
com_lease, traits Fila/Storage/Arquivos
│   └─ infra/               SQS, S3, repositório de arquivos, pool,
observabilidade
├ bins/
│   └─ worker/src/main.rs    main + run
│   └─ relay/src/main.rs     o relay da outbox, implantado separado
├ tests/                    integração com containers
└─ deploy/                  Dockerfile, docker-compose, k8s,
ScaledObject
```

Separar o `cnab` num crate sem dependências de rede ou banco tem duas vantagens: ele compila e testa em segundos, e o compilador garante que o parser não faz I/O escondido. As dependências principais do workspace: `tokio`, `tokio-util`, `async-channel`, `tokio-postgres` (features de `uuid` e `chrono`), `deadpool-postgres`, `aws-sdk-s3`, `aws-sdk-sqs`, `uuid` (feature `v7`), `chrono`, `thiserror`, `anyhow`, `tracing`, `tracing-subscriber`, `tracing-opentelemetry`, `mimalloc` e, nos testes e benchmarks, `criterion`, `proptest` e `testcontainers`.

Contratos

```
#[derive(Debug, Clone)]
pub struct Mensagem {
    pub arquivo_id: Uuid,
    pub chave_storage: String,
    pub banco: String,
    pub id_recebimento: String,          // o receipt handle do SQS
    pub atributos: HashMap<String, String>, // inclui o traceparent (Módulo
9.2)
}

pub trait Fila: Send + Sync + 'static {
    /// None quando o token é cancelado durante a espera
```

```

    fn proxima(&self, cancelamento: &CancellationToken) -> impl Future<Output
= Option<Mensagem>> + Send;
    fn confirmar(&self, m: &Mensagem) -> impl Future<Output = Result<(),
ErroDeFila>> + Send;
    fn estender_visibilidade(&self, m: &Mensagem, prazo: Duration) -> impl
Future<Output = Result<(), ErroDeFila>> + Send;
}

pub trait Storage: Send + Sync + 'static {
    fn abrir(&self, chave: &str)
        -> impl Future<Output = Result<Box<dyn AsyncRead + Send + Unpin>,
ErroDeStorage>> + Send;
}

pub trait Arquivos: Send + Sync + 'static {
    /// Some(tentativa) se assumiu; None se outro worker está com ele ou já
terminou
    fn tentar_assumir(&self, id: Uuid) -> impl Future<Output =
Result<Option<i32>, ErroDeArquivos>> + Send;
    fn renovar_lease(&self, id: Uuid) -> impl Future<Output = Result<(),
ErroDeArquivos>> + Send;
    fn reconciliar_e_concluir(&self, id: Uuid, c: &ContextoDoArquivo)
        -> impl Future<Output = Result<StatusArquivo, ErroDeArquivos>> +
Send;
    fn marcar_rejeitado(&self, id: Uuid, motivo: &str) -> impl Future<Output
= Result<(), ErroDeArquivos>> + Send;
    fn marcar_falha(&self, id: Uuid, motivo: &str) -> impl Future<Output =
Result<(), ErroDeArquivos>> + Send;
}

#[derive(Debug, thiserror::Error)]
pub enum ErroDeProcessamento {
    #[error("lease perdido: {0}")]
    LeasePerdido(ErroDeArquivos),
    #[error(transparent)]
    Storage(#[from] ErroDeStorage),
    #[error(transparent)]
    Pipeline(#[from] ErroDoPipeline),
}

```

O **Worker** é genérico sobre esses quatro traits (fila, storage, arquivos e escritor). Em produção ele é instanciado com SQS, S3, PostgreSQL e o escritor com retry; nos testes, com implementações em memória. O despacho é estático: o compilador gera uma versão especializada para cada combinação, sem chamadas virtuais no caminho quente.

Orun

```

async fn run(config: Config, cancelamento: CancellationToken) ->
anyhow::Result<()> {
    let _telemetria = observabilidade::iniciar(&config)?; // guarda:
    descarrega spans e métricas no Drop

    let pool = infra::banco::novo_pool(
        &config.banco_url,
        config.paralelo * config.escritores + 2, // max_size = A·E + X
        (Módulo 10.2)
    )?;
    let batimento = Arc::new(Batimento::default());
    tokio::spawn(infra::admin::servir(config.porta_admin, batimento.clone(),
    cancelamento.clone()));

    let worker = Arc::new(Worker {
        fila: Arc::new(infra::Sqs::nova(&config).await?),
        arquivos: Arc::new(infra::RepositorioDeArquivos::novo(pool.clone(),
        config.identidade.clone())),
        storage: Arc::new(infra::S3::novo(&config).await?),
        escritor:
        Arc::new(EscritorComRetry(EscritorPostgres::novo(pool.clone()))), // Módulos
        6 e 8
        opcoes: pipeline::Opcoes { capacidade_lotes: 4, escritores:
        config.escritores },
        paralelo: config.paralelo,
        metricas: Metricas::novas(),
        batimento,
    });

    tracing::info!(
        threads = tokio::runtime::Handle::current().metrics().num_workers(),
        "worker iniciado" // confirma que o limite de CPU do pod foi
        respeitado (Módulo 10.3)
    );
    worker.consumir(cancelamento).await; // até o SIGTERM, esperando os
    arquivos em andamento
    Ok(())
}

```

O processamento de uma mensagem

Cada braço do `match` final corresponde a uma decisão de algum módulo, e o compilador garante que nenhum caso ficou sem tratamento:

```

const MAX_TENTATIVAS: i32 = 5;

#[tracing::instrument(name = "processar-arquivo", skip_all, fields(arquivo_id
= %msg.arquivo_id))]
async fn processar_mensagem(&self, msg: Mensagem, cancelamento:
&CancellationToken) {
    self.batimento.marcar();
    let inicio = Instant::now();

    let tentativa = match self.arquivos.tentar_assumir(msg.arquivo_id).await
{
    Ok(Some(t)) => t,
    Ok(None) => {
        let _ = self.fila.confirmar(&msg).await; // outro worker está com
ele, ou já terminou
        return;
    }
    Err(e) => {
        tracing::error!(erro = %e, "não foi possível assumir o arquivo");
        return; // sem ack: a fila reentrega
    }
};
    if tentativa > MAX_TENTATIVAS {
        let _ = self.arquivos.marcar_falha(msg.arquivo_id, "tentativas
esgotadas").await;
        let _ = self.fila.confirmar(&msg).await;
        return;
    }

    let m = &msg;
    let resultado = self
        .com_lease(m, cancelamento, |token| async move {
            let origem = self.storage.abrir(&m.chave_storage).await?; //
stream (Módulo 4.4)
            let contexto = pipeline::processar_arquivo(
                origem, m.arquivo_id, self.escritor.clone(), &self.opcoes,
&token,
            ).await?; // canal + COPY (Módulos 5 e 6)
            Ok(contexto)
        })
        .await;

    match resultado {
        Ok(contexto) => match
self.arquivos.reconciliar_e_concluir(msg.arquivo_id, &contexto).await {
            Ok(status) => {

```

```

        let _ = self.fila.confirmar(&msg).await; // ack só depois do
commit final
        self.metricas.arquivo_finalizado(status, &msg.banco,
inicio.elapsed());
    }
    Err(e) => tracing::error!(erro = %e, "reconciliação"),
},

Err(ErroDeProcessamento::Pipeline(ErroDoPipeline::Leitura(ErroDeLeitura::LayoutInvalido(motivo)))) => {
    let _ = self.arquivos.marcas_rejeitado(msg.arquivo_id,
motivo).await;
    let _ = self.fila.confirmar(&msg).await; // erro permanente:
repetir não muda o conteúdo
}
    Err(ErroDeProcessamento::LeasePerdido(_)) => {
        tracing::warn!("lease perdido; quem assumiu termina o arquivo");
    }
    Err(_) if cancelamento.is_cancelled() => {} // desligando: sem ack, a
fila reentrega
    Err(e) => tracing::error!(erro = %e, "falha ao processar"), // sem
ack; após N recebimentos, DLQ
}
    self.batimento.marcas();
}

```

Dois detalhes de Rust neste método. O padrão aninhado

`ErroDeProcessamento::Pipeline(ErroDoPipeline::Leitura(ErroDeLeitura::LayoutInvalido(motivo)))` seleciona exatamente o erro permanente, atravessando três camadas de tipos, sem `errors.Is` nem `instanceof`. E `let m = &msg;` existe para que a closure `async move` capture uma referência (que é `Copy`) em vez de mover a mensagem, que ainda é usada depois. Como nos outros cursos, não há `retry` neste nível: `retries` transitórios ficam no `EscritorComRetry`, reentrega na fila e duplicatas no banco; este método só decide **quando confirmar a mensagem**.

As onze etapas

- ☐ **1. Fundação.** Workspace, `docker-compose` com PostgreSQL e LocalStack, migrations, `main` com sinais e `CancellationToken`. Critério: sobe, registra as threads do runtime e desliga limpo no SIGTERM.
- ☐ **2. Ingestão.** A API de um dos cursos de API da série ou um utilitário de linha de comando que envia ao S3, calcula o hash e publica. Critério: o mesmo arquivo enviado duas vezes gera um registro e uma mensagem.

- ☐ **3. Parser.** Crate `cnab` com testes de tabela, `proptest` e o teste do alocador contador. Critério: zero alocações por linha; casos de linha curta, data inválida, trailer divergente e última linha sem quebra.
- ☐ **4. Leitor e pipeline.** `leitor::ler` e `processar_arquivo` com um escritor falso. Critério: memória estável com escritor lento; erro no lote 10 chega como `Escrita`; `panic` num escritor volta como `Panico` sem derrubar o processo.
- ☐ **5. Escritor real.** `EscritorPostgres` com `COPY` binário e consolidação. Critério: 1 milhão de linhas gravadas; regravar não muda contagens nem gera eventos.
- ☐ **6. Lease.** `tentar_assumir` e `com_lease`. Critério: dois workers com a mesma mensagem, só um processa; lease perdido devolve `LeasePerdido`.
- ☐ **7. Resiliência.** `com_retry`, `Transitorio`, DLQ configurada. Critério: banco fora por 30 s não gera DLQ; layout inválido vai para `REJEITADO` na primeira tentativa.
- ☐ **8. Relay da outbox.** Binário `relay` com `SKIP LOCKED`. Critério: matar o relay no meio de um lote não duplica efeitos num consumidor com `inbox`.
- ☐ **9. Reconciliação.** `ContextoDoArquivo`, `Reconciliacao` e `reconciliar_e_concluir`. Critério: um bug proposital que descarta uma linha é detectado.
- ☐ **10. Observabilidade.** `tracing` com JSON e ponte OTLP, propagação do trace pela fila, métricas e gauges. Critério: gargalo identificado só pelas métricas; logs e traces com o mesmo `trace_id`.
- ☐ **11. Carga, caos e deploy.** Teste de carga, `kill -9` em momentos diferentes, imagem `musl` estática com `mimalloc`, Kubernetes com KEDA e limite de CPU. Critério: atender a ficha do Módulo 1 com margem e sobreviver a um rollout durante o pico.

Extensão opcional, depois da etapa 9: o fan-out em chunks, com `seek` + `take` num arquivo local ou `Range` no S3.

Como testar o worker

- **Unidade:** testes de tabela no crate `cnab` e testes de propriedade com `proptest` (para qualquer sequência de bytes, `parse_detalhe` nunca entra em `panic` e, quando aceita, a ida e volta dos campos numéricos é exata). Para fuzzing guiado por cobertura, o `cargo fuzz` usa o mesmo motor do `libFuzzer`.
- **Alocação:** o teste do alocador contador do Módulo 3.3, num binário de teste próprio.
- **Integração:** o crate `testcontainers` sobe PostgreSQL e LocalStack; o teste instancia o `Worker` com as implementações reais apontando para os containers, envia um arquivo, publica a mensagem e espera (com `tokio::time::timeout`) o status `CONCLUIDO`; depois confere `retorno_detalhe` e `outbox`. O mesmo teste processa o arquivo duas vezes para verificar a idempotência.

- **Tempo:** com a feature `test-util` do tokio, `#[tokio::test(start_paused = true)]` congela o relógio e `tokio::time::advance` o avança, o que testa o backoff do `com_retry` e a renovação do lease sem esperar de verdade (o equivalente do `testing/syncctest` do Go).
- **Qualidade:** `cargo clippy -- -D warnings` e `cargo audit` (base de vulnerabilidades RustSec) no CI.

CrITÉRIOS de aceite finais

- ☐ Vazão sustentada $\geq$ meta de pico da ficha, com 2× de margem
- ☐ O maior arquivo previsto termina dentro do SLA
- ☐ Memória de cada réplica estável durante o maior arquivo, e menor que a dos workers em .NET e Go no mesmo teste
- ☐ Zero alocações por linha no parser, provado por teste
- ☐ Reprocessar qualquer arquivo não altera contagens nem gera eventos
- ☐ `kill -9` em qualquer momento não perde nem duplica registros
- ☐ Toda divergência com o trailer gera alerta
- ☐ Nenhum `unsafe` fora do teste do alocador; `clippy` e `cargo audit` limpos
- ☐ Workers em Rust, Go e .NET dividindo a mesma fila produzem o mesmo resultado

Referências e próximos passos

Com este curso, o pipeline de arquivos existe em três linguagens sobre o mesmo banco e as mesmas mensagens. O exercício mais revelador da série agora é rodar os três workers no mesmo teste de carga e comparar não só a vazão (que o PostgreSQL deve igualar) mas memória por réplica, estabilidade da latência de cada lote, tempo de subida, tamanho da imagem e, principalmente, quantos bugs de concorrência cada linguagem deixou chegar ao teste de caos. Outros próximos passos naturais em Rust: uma API HTTP com `axum` sobre o mesmo domínio, equivalente aos cursos de API em .NET e Go, e o crate `cnab` publicado como biblioteca reutilizável.

Os outros cursos da série

- [Curso: .NET 10 e System Design para processamento de arquivos em alto volume](#)
- [Curso: Go e System Design para processamento de arquivos em alto volume](#)
- [Curso: Minimal APIs no ASP.NET Core 10](#)
- [Curso: APIs HTTP em Go](#)

Documentação por módulo

Módulo	Leitura
3. Memória	The Rust Programming Language: ownership · The Rust Performance Book
3 a 5. Runtime e concorrência	Tutorial do tokio · tokio-util: CancellationToken e TaskTracker · async-channel
6 e 7. Banco	tokio-postgres · deadpool-postgres · sqlx · PostgreSQL: carga em massa
8. Confiabilidade	thiserror · anyhow · Padrão outbox transacional
9. Observabilidade	tracing · tokio-console · criterion
10. Deploy	Cargo: perfis de compilação · KEDA: scaler de AWS SQS

Livros

- *Rust for Rustaceans*, Jon Gjengset: tipos, traits, `async` e `unsafe` no nível que este curso pressupõe depois do Módulo 3.
- *Rust Atomics and Locks*, Mara Bos: concorrência de baixo nível, para entender o que `Send`, `Sync` e os canais garantem por baixo.
- *Zero To Production In Rust*, Luca Palmieri: um serviço backend completo em Rust, com testes, observabilidade e deploy.
- *Designing Data-Intensive Applications*, Martin Kleppmann: a base teórica comum a toda a série.

Fonte sobre versões

- [Anúncios de versão do Rust](#): Rust 1.99.0, lançado em 1º de outubro de 2026, é a versão estável atual; a edição 2024 chegou com o Rust 1.85, em fevereiro de 2025.